Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 18

Arquitecturas modernas

Normalización, atención eficiente, contexto largo, mezcla de expertos, modelos de espacio de estados y multimodalidad

---

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

El Transformer de los módulos 13 a 15 gasta el mismo cómputo en cada token, guarda durante la generación las claves y los valores de todas las posiciones anteriores y solo procesa texto. Los modelos de lenguaje publicados desde 2023 conservan su estructura, pero modifican casi todas sus piezas para reducir el coste por token, la memoria de inferencia o la dependencia cuadrática de la longitud del contexto. En marzo de 2024 xAI publicó los pesos de Grok-1, un modelo de unos $3{,}1 \cdot 10^{11}$ parámetros que, sin embargo, usa solo una cuarta parte de ellos en cada token; el informe técnico de Gemini 1.5 lo describe como un Transformer de mezcla de expertos dispersa que procesa contextos de millones de tokens con texto, audio y vídeo intercalados. Este módulo estudia, con la notación y el rigor de los anteriores, los mecanismos que hacen posibles esas cifras: las componentes de consenso del bloque moderno, las variantes de la atención que reducen la caché de claves y valores, la atención local y la extensión del contexto, la mezcla de expertos, los modelos de espacio de estados y la representación de imágenes y audio como tokens. Sobre Claude, Gemini y Grok solo se usa información pública; cuando un dato no lo es, se indica.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. describir el bloque Transformer de consenso (pre-norm, RMSNorm, SwiGLU, RoPE, sin sesgos), demostrar las propiedades de invariancia de RMSNorm y justificar los hiperparámetros de forma habituales;
2. calcular la memoria de la caché de claves y valores con MHA, GQA, MQA y MLA, demostrar que la decodificación con MHA tiene intensidad aritmética constante y explicar la absorción de matrices de MLA;
3. definir la atención con ventana deslizante, demostrar la fórmula de su campo receptivo y la corrección de la caché circular, y analizar la interpolación de posiciones, el cambio de base NTK y YaRN para extender el contexto de RoPE;
4. formular una capa de mezcla de expertos con enrutado top-k, distinguir parámetros totales y activos, demostrar las cotas de la pérdida de balanceo y reconstruir los recuentos publicados de Mixtral y Grok-1;
5. discretizar un modelo de espacio de estados, demostrar la equivalencia entre su forma recurrente y convolucional, la conexión del modelo selectivo de Mamba con una puerta recurrente y la asociatividad que permite el barrido paralelo;
6. explicar cómo un Vision Transformer y un modelo de voz convierten imágenes y audio en secuencias de vectores y calcular el número de tokens y el coste resultantes.

Requisitos previos

Del módulo 2, el producto de matrices y la factorización de bajo rango; del módulo 4, la regla de la cadena; del módulo 5, la desigualdad de Cauchy–Schwarz; del módulo 11, LayerNorm, RMSNorm y las conexiones residuales; del módulo 12, las redes recurrentes y sus puertas; del módulo 13, la atención escalada, la máscara causal y la atención multicabeza; del módulo 14, el bloque decoder-only, SwiGLU, RoPE (tema 4) y el recuento de parámetros y FLOPs; del módulo 16, la regla $C \approx 6ND$, y del módulo 17, el modelo *roofline* y la intensidad crítica de la A100, $I^* \approx 156$ FLOP/byte en bf16. La caché de claves y valores se introduce aquí y se implementa en el módulo 19.

Organización

Tema	Contenido	Fuentes principales
1	Componentes de consenso del Transformer moderno	[16], [33], [25], [13]
2	Atención eficiente en memoria: caché KV, MQA, GQA y MLA	[24], [1], [6], [16]
3	Atención local y contexto largo	[19], [2], [3], [22]
4	Mezcla de expertos	[26], [10], [18], [7]
5	Modelos de espacio de estados	[15], [14], [21], [11]
6	Multimodalidad: imágenes y audio como tokens	[9], [23], [31], [32]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: ⋆ comprensión, ⋆⋆ cálculo y ⋆⋆⋆ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La organización sigue el bloque de arquitecturas del curso CS336 de Stanford, «Language Modeling from Scratch» [16]: la clase 3 sobre arquitecturas e hiperparámetros, la clase 4 sobre mezcla de expertos y la parte de la clase 10 (inferencia) dedicada a reducir la caché de claves y valores y a las alternativas al Transformer. Se complementa con el seminario CS25 de Stanford [11], en particular la sesión de A. Gu sobre modelos de espacio de estados, y con los temarios de los cursos CMU 11-711 (*Advanced NLP*) [4] y CMU 11-868 (*Large Language Model Systems*) [5]. Los métodos se toman de sus artículos originales: Zhang y Sennrich [33] para RMSNorm, Shazeer [25] para las unidades con puerta, Su *et al.* [27] para RoPE, Shazeer [24] para MQA, Ainslie *et al.* [1] para GQA, DeepSeek-AI [6, 7] para MLA y DeepSeekMoE, Jiang *et al.* [19] para la ventana deslizante, Beltagy *et al.* [2] para Longformer, Chen *et al.* [3] y Peng *et al.* [22] para la extensión del contexto, Shazeer *et al.* [26], Fedus *et al.* [10] y Jiang *et al.* [18] para la mezcla de expertos, Gu *et al.* [15] y Gu y Dao [14] para los modelos de espacio de estados, Dosovitskiy *et al.* [9] para el Vision Transformer y Radford *et al.* [23] para Whisper. Las configuraciones de Llama 2 y Llama 3 proceden de sus informes técnicos [28, 13], y las de Grok-1, del archivo `README` y del código publicados por xAI [30]. Los ejercicios universitarios proceden de la tarea 1 y de las clases de CS336 [17, 16], del curso 6.003 del MIT [21] y del libro *Dive into Deep Learning* [32]. Sobre

Claude no se usa ningún dato de arquitectura, porque Anthropic no lo ha publicado; sobre Gemini, solo el informe técnico [12], y sobre Grok, solo la versión de pesos abiertos [30].

Notación

Símbolo	Significado
d, L, d_{ff}	dimensión del modelo, número de bloques y dimensión intermedia del MLP
H, $d_h = d/H$	cabezas de consulta y dimensión por cabeza
n_{kv}, G	cabezas de clave y valor; número de grupos de GQA ($n_{\text{kv}} = G$)
T, b, s	longitud de la secuencia, tamaño del lote y bytes por número (2 en bf16)
d_c, d_h^R	dimensión latente de MLA y dimensión de la clave con RoPE desacoplada
W	tamaño de la ventana de atención local
θ_i, $\lambda_i = 2\pi/\theta_i$, β_0	frecuencias y longitudes de onda de RoPE; base (10^4 en el artículo original)
$s_{\text{e}} = L'/L_0$	factor de extensión del contexto, de L_0 a L' tokens
$E_1, \ldots, E_N$, k	expertos y número de expertos activos por token
$z = xW_r$, $p_i(x)$, $g_i(x)$	logits del enrutador, probabilidades y pesos de combinación
f_i, P_i, $\mathcal{L}_{\text{bal}}$	fracción de tráfico, probabilidad media y pérdida de balanceo del experto i
N_{tot}, N_{act}	parámetros totales y activos por token
A, B, C, Δ	matrices de un modelo de espacio de estados continuo y paso de discretización
$\bar{A}, \bar{B}$, h_t	matrices discretizadas y estado en el instante t
σ, $\text{softplus}(z) = \ln(1 + e^z)$	sigmoide logística y función *softplus*
P, C_{in}	lado del parche de un ViT y número de canales de la imagen

Como en el módulo 14, los vectores de tokens son filas y una capa lineal actúa como $x \mapsto xW$. Los números grandes se escriben con espacio fino como separador de miles (46 702 792 704). Las unidades binarias son 1 KiB $= 2^{10}$, 1 MiB $= 2^{20}$ y 1 GiB $= 2^{30}$ bytes; 1 GB son 10^9 bytes.

Índice

Presentación del módulo **ii**

1 Componentes de consenso del Transformer moderno **1**
1.1 El bloque de consenso . . . 1
1.2 RMSNorm . . . 2
1.3 El presupuesto del MLP con puerta . . . 3
1.4 Hiperparámetros de forma . . . 3
Ejercicios del tema 1 . . . 5

2 Atención eficiente en memoria: caché KV, MQA, GQA y MLA **6**
2.1 La caché de claves y valores . . . 6
2.2 La decodificación está limitada por la memoria . . . 7
2.3 Atención multiconsulta y de consultas agrupadas . . . 7
2.4 Atención latente multicabeza . . . 8
Ejercicios del tema 2 . . . 10

3 Atención local y contexto largo **11**
3.1 Atención con ventana deslizante . . . 11
3.2 Campo receptivo . . . 11
3.3 La caché circular . . . 12
3.4 Patrones dispersos e híbridos . . . 12
3.5 RoPE y longitudes de onda . . . 13
3.6 Extensión del contexto . . . 13
Ejercicios del tema 3 . . . 15

4 Mezcla de expertos **16**
4.1 La capa de mezcla de expertos . . . 16
4.2 Parámetros totales y activos . . . 17
4.3 Balanceo de carga . . . 18
4.4 Capacidad y descarte de tokens . . . 19
4.5 Expertos de grano fino y compartidos . . . 19
4.6 Casos públicos . . . 20
Ejercicios del tema 4 . . . 20

5 Modelos de espacio de estados **22**
5.1 Sistemas lineales continuos y su discretización . . . 22
5.2 Recurrencia y convolución . . . 23
5.3 Selección: el modelo de Mamba . . . 23
5.4 El barrido paralelo . . . 24
5.5 Comparación con la atención . . . 25
Ejercicios del tema 5 . . . 25

6 Multimodalidad: imágenes y audio como tokens **27**
6.1 El Vision Transformer . . . 27
6.2 Audio . . . 28
6.3 Fusión con el modelo de lenguaje . . . 29
Ejercicios del tema 6 . . . 29

A Soluciones de los ejercicios **31**
A.1 Tema 1 . . . 31
A.2 Tema 2 . . . 34
A.3 Tema 3 . . . 37
A.4 Tema 4 . . . 40
A.5 Tema 5 . . . 43
A.6 Tema 6 . . . 46

Bibliografía **49**

Tema 1

Componentes de consenso del Transformer moderno

Entre el Transformer original de 2017 y los modelos abiertos actuales hay decenas de variantes publicadas, pero la mayoría converge en un mismo bloque: normalización antes de cada subcapa, RMSNorm, MLP con puerta, codificación rotatoria de la posición y ausencia de sesgos. Este tema define ese bloque de consenso, demuestra las propiedades de RMSNorm que lo justifican y estudia los hiperparámetros de forma que comparten casi todos los modelos. Se sigue la clase 3 del curso CS336 [16], el artículo de Zhang y Sennrich [33, §3–4], el de Shazeer [25, §2–3] y la tabla de hiperparámetros de Llama 3 [13, §3.2]. El bloque decoder-only, SwiGLU y RoPE ya se han estudiado en el módulo 14; aquí se recuerdan solo en la medida en que se necesitan.

1.1 El bloque de consenso

La clase 3 de CS336 compara las arquitecturas densas publicadas entre 2019 y 2025 y observa que hay poco consenso en los detalles, con una excepción casi universal: la normalización se coloca a la entrada de cada subcapa, fuera del camino residual [16]. Las demás elecciones convergen hacia lo que el curso llama arquitecturas «de tipo LLaMA».

Definición 1.1 (Bloque de consenso). Sean $h \in \mathbb{R}^{1\times d}$ la fila de un token en el flujo residual, Attn una atención multicabeza causal con RoPE y FFN un MLP SwiGLU sin sesgos. El *bloque de consenso* es

$$a = h + \mathrm{Attn}\big(\mathrm{RMSNorm}_1(h)\big), \qquad h' = a + \mathrm{FFN}\big(\mathrm{RMSNorm}_2(a)\big),$$

donde las capas lineales no tienen sesgo y la posición se introduce solo mediante la rotación de consultas y claves en cada capa. El modelo completo añade un RMSNorm final antes de la matriz de salida.

Cuadro 1.1: Elecciones de arquitectura del Transformer original y de Llama 3. Fuentes: [29, §3], [13, §3.2 y tabla 3] y [16].

Elección	Transformer (2017)	Llama 3 (2024)
Posición de la normalización	tras la subcapa (post-norm)	antes de la subcapa (pre-norm)
Normalización	LayerNorm	RMSNorm
Activación del MLP	ReLU, $d_{\mathrm{ff}} = 4d$	SwiGLU, $d_{\mathrm{ff}} \approx 3{,}5d$
Posición	sinusoidal sumada a la entrada	RoPE en cada capa, base $5 \cdot 10^5$
Sesgos	sí	no
Atención	multicabeza (MHA)	agrupada (GQA, 8 cabezas K/V)

Observación 1.2. Las justificaciones de cada elección son mayoritariamente empíricas. Shazeer termina su artículo sobre las unidades con puerta sin ofrecer explicación de por qué funcionan mejor [25, §4], y la clase 3 de CS336 insiste en que las diferencias entre variantes son pequeñas y en que el argumento a favor de RMSNorm es sobre todo de coste [16]. Lo que sí admite demostración son las propiedades algebraicas que diferencian unas variantes de otras, y a ellas se dedica la sección siguiente.

1.2 RMSNorm

Definición 1.3 (LayerNorm y RMSNorm). Sean $x \in \mathbb{R}^d$, $\gamma \in \mathbb{R}^d$ un vector de ganancias y $\varepsilon \geq 0$. Con $\mu(x) = \frac{1}{d}\sum_i x_i$ y $\mathrm{RMS}_\varepsilon(x) = \sqrt{\frac{1}{d}\sum_i x_i^2 + \varepsilon}$, se definen

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}_\varepsilon(x)} \odot \gamma, \qquad \mathrm{LN}(x) = \frac{x - \mu(x)\mathbf{1}}{\sqrt{\frac{1}{d}\sum_i (x_i - \mu(x))^2 + \varepsilon}} \odot \gamma + \beta,$$

donde $\odot$ es el producto componente a componente y $\beta \in \mathbb{R}^d$ es el sesgo de LayerNorm. RMSNorm no resta la media ni tiene sesgo [33, §4, ec. 4].

La práctica 1 de CS336 usa exactamente esta forma, con $\varepsilon = 10^{-5}$ y cálculo en fp32 aunque la entrada esté en bf16, para evitar desbordamientos al elevar al cuadrado [17, §3.4.1].

Proposición 1.4 (Propiedades de RMSNorm). *Sea $\varepsilon = 0$ y sea $x \neq 0$.*

(i) Norma fija. *Con $\gamma = \mathbf{1}$,* $\|\mathrm{RMSNorm}(x)\| = \sqrt{d}$.

(ii) Invariancia de escala. *Para todo $\alpha > 0$,* $\mathrm{RMSNorm}(\alpha x) = \mathrm{RMSNorm}(x)$*; para $\alpha < 0$,* $\mathrm{RMSNorm}(\alpha x) = -\mathrm{RMSNorm}(x)$.

(iii) Invariancia ante la escala de los pesos. *Si $x = uW$ es la salida de una capa lineal y W se sustituye por αW con $\alpha > 0$, la salida normalizada no cambia.*

(iv) Relación con LayerNorm. *Con $\beta = 0$,* $\mathrm{LN}(x) = \mathrm{RMSNorm}\big(x - \mu(x)\mathbf{1}\big)$ *siempre que x no sea constante.*

Demostración. (i) $\|x/\,\mathrm{RMS}(x)\|^2 = \sum_i x_i^2 \big/ \big(\frac{1}{d}\sum_i x_i^2\big) = d$.

(ii) $\mathrm{RMS}(\alpha x) = \sqrt{\frac{1}{d}\sum_i \alpha^2 x_i^2} = |\alpha|\,\mathrm{RMS}(x)$, luego $\alpha x/\,\mathrm{RMS}(\alpha x) = (\alpha/|\alpha|)\, x/\,\mathrm{RMS}(x)$, y $\alpha/|\alpha| = \pm 1$ según el signo de α.

(iii) $u(\alpha W) = \alpha\,(uW) = \alpha x$, y basta aplicar (ii).

(iv) Sea $c = x - \mu(x)\mathbf{1}$, que no es nulo porque x no es constante. Como $\mu(c) = 0$, el denominador de LayerNorm es $\sqrt{\frac{1}{d}\sum_i c_i^2} = \mathrm{RMS}(c)$, y $\mathrm{LN}(x) = c/\,\mathrm{RMS}(c) \odot \gamma = \mathrm{RMSNorm}(c)$. ∎

La proposición 1.4(iv) dice que LayerNorm es RMSNorm precedida de un centrado. Lo que RMSNorm pierde es, por tanto, la invariancia ante traslaciones: $\mathrm{LN}(x + t\mathbf{1}) = \mathrm{LN}(x)$ para todo $t \in \mathbb{R}$, mientras que $\mathrm{RMSNorm}(x + t\mathbf{1}) \neq \mathrm{RMSNorm}(x)$ en general. Zhang y Sennrich conjeturan que la invariancia ante la escala es la propiedad que importa para la estabilidad, y no la de centrado, y la tabla 1 de su artículo compara ambas normalizaciones en estos términos [33, §4.1].

Ejemplo 1.5 (RMSNorm frente a LayerNorm). Sea $x = (1, 2, 3, -2)$ con $\gamma = \mathbf{1}$, $\beta = 0$ y $\varepsilon = 0$. Entonces $\sum_i x_i^2 = 18$, $\mathrm{RMS}(x) = \sqrt{18/4} = 2{,}121$ y

$$\mathrm{RMSNorm}(x) = (0{,}4714;\ 0{,}9428;\ 1{,}414;\ -0{,}9428),$$

cuya norma es $2 = \sqrt{4}$. Para LayerNorm, $\mu(x) = 1$, el vector centrado es $c = (0, 1, 2, -3)$, con $\sum_i c_i^2 = 14$ y $\mathrm{RMS}(c) = \sqrt{3{,}5} = 1{,}871$, y

$$\mathrm{LN}(x) = (0;\ 0{,}5345;\ 1{,}069;\ -1{,}604).$$

Las dos salidas difieren porque la media de x no es nula; si se duplica x, ninguna de las dos cambia.

Observación 1.6 (Coste). Las normalizaciones suponen una fracción despreciable de los FLOPs de un bloque, pero no de su tiempo: son operaciones limitadas por la memoria, con intensidad aritmética baja (módulo 17, tema 2). Por eso la clase 3 de CS336 atribuye a RMSNorm una ganancia de tiempo real apreciable a pesar de ahorrar muy pocas operaciones [16], y Zhang y Sennrich informan de reducciones del tiempo de ejecución de entre el 7 y el 64 % según el modelo [33].

1.3 El presupuesto del MLP con puerta

Definición 1.7 (MLP SwiGLU). Sean $W_1, W_3 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$ y $W_2 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$. El MLP SwiGLU es

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \big(\mathrm{SiLU}(xW_1) \odot xW_3\big)W_2, \qquad \mathrm{SiLU}(t) = t\,\sigma(t),$$

sin sesgos [25, ec. 6].

Proposición 1.8 (Presupuesto de parámetros). *Un MLP SwiGLU con dimensión intermedia d_{ff} tiene $3d\,d_{\mathrm{ff}}$ parámetros y cuesta $6d\,d_{\mathrm{ff}}$ FLOPs por token en la pasada hacia delante. Tiene los mismos parámetros y FLOPs que un MLP de dos matrices con dimensión intermedia $4d$ si y solo si $d_{\mathrm{ff}} = \frac{8}{3}d$.*

Demostración. Cada una de las tres matrices tiene $d\,d_{\mathrm{ff}}$ entradas, y cada producto de una fila por una matriz $d \times d_{\mathrm{ff}}$ (o $d_{\mathrm{ff}} \times d$) cuesta $2d\,d_{\mathrm{ff}}$ FLOPs (módulo 14, tema 6). El MLP de dos matrices con $4d$ tiene $2 \cdot d \cdot 4d = 8d^2$ parámetros. La igualdad $3d\,d_{\mathrm{ff}} = 8d^2$ equivale a $d_{\mathrm{ff}} = 8d/3$; el argumento para los FLOPs es idéntico, con un factor 2 en ambos miembros. ∎

Shazeer aplica esta regla en sus experimentos: compara un MLP con $d_{\mathrm{ff}} = 3072$ y dos matrices con variantes con puerta de $d_{\mathrm{ff}} = 2048 = \frac{2}{3} \cdot 3072$ [25, §3.1]. La clase 3 de CS336 recoge que la mayoría de los modelos con puerta usan cocientes d_{ff}/d entre 2,5 y 3,5, y que fuera de ese intervalo hay excepciones como T5, con un cociente de 64 [16]. La cuadro 1.2 muestra los valores de Llama 3, Mixtral y Grok-1.

1.4 Hiperparámetros de forma

La clase 3 de CS336 identifica tres regularidades más, todas ellas con excepciones documentadas [16].

Definición 1.9 (Hiperparámetros de forma). En un Transformer con L bloques, dimensión d y H cabezas de dimensión d_h:

Cuadro 1.2: Hiperparámetros de forma de modelos con publicación de su configuración. Fuentes: [13, tabla 3], [18, tabla 1] y el archivo `run.py` de [30].

Modelo	L	d	d_{ff}	H	n_{kv}	d_{ff}/d	d/L
Llama 3 8B	32	4096	14 336	32	8	3,5	128
Llama 3 70B	80	8192	28 672	64	8	3,5	102,4
Llama 3 405B	126	16 384	53 248	128	8	3,25	130,0
Mixtral 8x7B	32	4096	14 336	32	8	3,5	128
Grok-1	64	6144	32 768	48	8	5,333	96

(i) la *regla de las cabezas* es $Hd_h = d$;

(ii) la *relación de aspecto* es $r = d/L$;

(iii) el *tamaño del vocabulario* es $|\mathcal{V}|$.

Los modelos de la cuadro 1.2 cumplen la regla de las cabezas con $d_h = 128$, salvo Grok-1, en el que $48 \cdot 128 = 6144 = d$ también la cumple. La clase de CS336 sitúa la relación de aspecto de la mayoría de los modelos entre 100 y 200, y los vocabularios entre $3 \cdot 10^4$ y $5 \cdot 10^4$ tokens para modelos monolingües y entre 10^5 y $2{,}5 \cdot 10^5$ para los multilingües: Llama 2 usa 32 000 y Llama 3, unos 128 000 [16, 13].

Proposición 1.10 (Dimensión a partir del tamaño y la relación de aspecto). *Sea un Transformer de consenso con $d_{\text{ff}} = \frac{8}{3}d$ y atención multicabeza, cuyos bloques contienen N_{b} parámetros en total, y sea $r = d/L$ su relación de aspecto. Despreciando las normalizaciones, $N_{\text{b}} = 12Ld^2$ y, por tanto,*

$$d = \left(\frac{rN_{\text{b}}}{12}\right)^{1/3}, \qquad L = \frac{d}{r}.$$

Demostración. La atención tiene cuatro matrices $d \times d$, es decir, $4d^2$ parámetros; el MLP, $3d\,d_{\text{ff}} = 8d^2$ por la proposición 1.8. Cada bloque tiene $12d^2$ parámetros y L bloques, $12Ld^2$. Sustituyendo $L = d/r$, $N_{\text{b}} = 12d^3/r$, y se despeja d. ∎

Ejemplo 1.11 (Forma de un modelo de $4 \cdot 10^{11}$ parámetros). Con $N_{\text{b}} = 4{,}05 \cdot 10^{11}$ y $r = 130$, la proposición 1.10 da $d = (130 \cdot 4{,}05 \cdot 10^{11}/12)^{1/3} = (4{,}388 \cdot 10^{12})^{1/3} = 16\,371$ y $L = 16\,371/130 = 125{,}9$. Llama 3 405B tiene $d = 16\,384$ y $L = 126$ (cuadro 1.2). La coincidencia es en parte casual, porque Llama 3 usa GQA y $d_{\text{ff}} = 3{,}25d$, pero muestra que dos números bastan para fijar la forma de un modelo.

Observación 1.12 (Estabilidad). La clase 3 de CS336 recoge tres recursos de estabilidad frecuentes en modelos recientes, todos dirigidos a las funciones softmax: la pérdida auxiliar z sobre el logaritmo del denominador de la softmax de salida, la normalización de consultas y claves antes del producto escalar (*QK-norm*) y el recorte suave de los logits mediante una tangente hiperbólica [16]. Las tres limitan el crecimiento de los logits, cuyo exceso satura la softmax y desestabiliza el entrenamiento.

Lecturas recomendadas para este tema

- Stanford CS336 [16], clase 3, «Architectures, hyperparameters»: comparación de arquitecturas y tablas de hiperparámetros.
- Zhang y Sennrich [33], §4 «RMSNorm» y §4.1 «Invariance Analysis».

– Shazeer [25], §2 «Gated Linear Units (GLU) and Variants» y §3.1.

– Grattafiori *et al.* [13], §3.2 «Model Architecture» y tabla 3.

– Tarea 1 de CS336 [17], §3.4.1 (RMSNorm) y §7.3 (ablaciones de arquitectura).

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Justifique si son verdaderas o falsas: (a) RMSNorm es invariante ante la suma de una constante a todas las componentes; (b) un MLP SwiGLU con $d_{\text{ff}} = 4d$ tiene un 50 % más de parámetros que un MLP ReLU con $d_{\text{ff}} = 4d$; (c) en todos los modelos de la cuadro 1.2 se cumple $Hd_h = d$.

Ejercicio 1.2 ⋆⋆ Sea $x = (2, -1, 0, 3)$, con $\gamma = \mathbf{1}$, $\beta = 0$ y $\varepsilon = 0$. Calcule $\mathrm{RMSNorm}(x)$, $\mathrm{LN}(x)$ y $\mathrm{RMSNorm}(3x)$, y compruebe la proposición 1.4(i).

Ejercicio 1.3 ⋆⋆ Con los datos de la cuadro 1.2, calcule para los tres modelos de Llama 3 la dimensión por cabeza, el número de cabezas de consulta que comparten cada cabeza de clave y valor, y los parámetros de un bloque (atención con GQA, SwiGLU y dos RMSNorm). ¿Qué fracción del bloque corresponde al MLP en el modelo 8B?

Ejercicio 1.4 ⋆⋆ En la ablación de la tarea 1 de CS336 se compara un MLP SwiGLU con $d_{\text{ff}} \approx \frac{8}{3}d$, redondeado a un múltiplo de 64, con un MLP de dos matrices con activación SiLU y $d_{\text{ff}} = 4d$. Con $d = 512$: (a) calcule el d_{ff} del SwiGLU redondeando $\frac{8}{3}d$ al múltiplo de 64 más próximo; (b) cuente los parámetros de ambos MLP; (c) calcule la diferencia relativa y explique por qué la comparación es justa.

Fuente: adaptado de la tarea 1 de CS336 [17], §7.3, problema `swiglu_ablation`.

Ejercicio 1.5 ⋆⋆⋆ Demuestre que $\mathrm{LN}(x + t\mathbf{1}) = \mathrm{LN}(x)$ para todo $t \in \mathbb{R}$ y toda x no constante, y que RMSNorm no tiene esta propiedad: encuentre $x \in \mathbb{R}^2$ y t con $\mathrm{RMSNorm}(x + t\mathbf{1}) \neq \mathrm{RMSNorm}(x)$. Demuestre también que, si $\gamma = \mathbf{1}$, la salida de RMSNorm determina x salvo un factor positivo.

Ejercicio 1.6 ⋆⋆⋆ (a) Generalice la proposición 1.10 a un MLP con $d_{\text{ff}} = \rho d$ y atención con n_{kv} cabezas de clave y valor de dimensión $d_h = d/H$. (b) Aplique la fórmula de la proposición 1.10 a $N_{\text{b}} = 7 \cdot 10^{10}$ y $r = 102$, y compare con Llama 3 70B. (c) Recalcule los parámetros de los bloques de Llama 3 70B con la fórmula de (a) y explique la diferencia con (b).

Tema 2

Atención eficiente en memoria: caché KV, MQA, GQA y MLA

Al generar texto, un Transformer produce un token cada vez y reutiliza las claves y los valores de las posiciones anteriores. Esa reutilización ahorra cómputo, pero obliga a guardar y a leer en cada paso una caché que crece con la longitud del contexto y con el tamaño del lote, y que en contextos largos ocupa más memoria que los propios pesos. Este tema cuantifica esa caché, demuestra que con atención multicabeza la decodificación está necesariamente limitada por la memoria y estudia las tres variantes que la reducen: la atención multiconsulta (MQA), la atención de consultas agrupadas (GQA) y la atención latente multicabeza (MLA). Se sigue a Shazeer [24, §2.4 y §3], a Ainslie *et al.* [1, §2], a DeepSeek-AI [6, §2.1] y la clase 10 de CS336 [16].

2.1 La caché de claves y valores

Sea una capa de atención con H cabezas de dimensión d_h. En la generación autorregresiva, al procesar el token t se necesitan las consultas de t y las claves y los valores de las posiciones $1, \dots, t$. Por la causalidad (módulo 14, tema 1), las claves y los valores de las posiciones $j < t$ no dependen de t: se calculan una vez y se guardan.

Definición 2.1 (Caché KV). La *caché KV* de una capa, tras procesar T tokens, es el par de tensores (K, V) con las claves y los valores de las T posiciones para cada una de las n_{kv} cabezas de clave y valor. Cada tensor tiene $T \cdot n_{\text{kv}} \cdot d_h$ números por secuencia.

Proposición 2.2 (Tamaño de la caché KV). *Un modelo de L capas con n_{kv} cabezas de clave y valor de dimensión d_h, que procesa un lote de b secuencias de T tokens con s bytes por número, necesita*

$$M_{\text{KV}} = 2\, L\, n_{\text{kv}}\, d_h\, T\, b\, s \ \text{bytes}.$$

Demostración. Por la definición 2.1, cada capa guarda $T n_{\text{kv}} d_h$ números para K y otros tantos para V, por secuencia. Se multiplica por 2 (claves y valores), por L capas, por b secuencias y por s bytes. ∎

Ejemplo 2.3 (Llama 2 70B). Llama 2 70B tiene $L = 80$, $H = 64$ cabezas de consulta de dimensión $d_h = 128$ y usa GQA con $n_{\text{kv}} = 8$ [28]. En bf16 ($s = 2$), por token y secuencia, la caché ocuparía $2 \cdot 80 \cdot 64 \cdot 128 \cdot 2 = 2\,621\,440$ bytes = 2,5 MiB con atención multicabeza ($n_{\text{kv}} = 64$) y ocupa 327 680 bytes = 320 KiB con GQA. Para $T = 4096$ y $b = 16$, eso son 160 GiB frente a 20 GiB. Los pesos en bf16 ocupan unos 140 GB.

2.2 La decodificación está limitada por la memoria

El módulo 17 definió la intensidad aritmética de una operación como el cociente entre FLOPs y bytes transferidos, y la comparó con la intensidad crítica del acelerador. El paso de atención de la decodificación tiene una intensidad que no mejora con el lote.

Proposición 2.4 (Intensidad de la atención en la decodificación). *Sea una capa con H cabezas de consulta y n_{kv} cabezas de clave y valor de dimensión d_h, y sea un paso de decodificación de una secuencia cuyo contexto tiene T posiciones. Contando solo los productos $q\,K^\top$ y $a\,V$ y la lectura de la caché, la intensidad aritmética es*

$$I = \frac{2H}{s\,n_{\text{kv}}}\ \textit{FLOP/byte},$$

independiente de T y del tamaño del lote. Con MHA ($n_{\text{kv}} = H$) en bf16, $I = 1$.

Demostración. Para cada cabeza de consulta, el producto de $q \in \mathbb{R}^{1\times d_h}$ por $K^\top \in \mathbb{R}^{d_h\times T}$ cuesta $2d_hT$ FLOPs, y el de los pesos $a \in \mathbb{R}^{1\times T}$ por $V \in \mathbb{R}^{T\times d_h}$, otros $2Td_h$. Con H cabezas: $F = 4HTd_h$. Los bytes leídos de la caché son $Q = 2Tn_{\text{kv}}d_hs$, porque cada cabeza de clave y valor se lee una vez y la comparten las H/n_{kv} consultas de su grupo. Entonces $I = F/Q = 4HTd_h/(2Tn_{\text{kv}}d_hs) = 2H/(sn_{\text{kv}})$. Con un lote de b secuencias, F y Q se multiplican por b, porque cada secuencia tiene su propia caché, y el cociente no cambia. ■

La intensidad crítica de la A100 en bf16 es $I^* \approx 156$ FLOP/byte (módulo 17, tema 2). Con MHA, la atención de la decodificación aprovecha menos del 1 % del cómputo disponible, y aumentar el lote no ayuda. Los productos de las matrices de pesos, en cambio, sí ganan intensidad con el lote, porque todas las secuencias comparten los mismos pesos. Shazeer llega a la misma conclusión con otro recuento: en la decodificación incremental con MHA, el cociente entre accesos a memoria y operaciones es $\Theta(n/d + 1/b)$, y el término n/d, debido a la lectura de las claves y los valores, no disminuye con el lote [24, §2.4.1]. Reducir n_{kv} es la forma directa de atacarlo.

2.3 Atención multiconsulta y de consultas agrupadas

Definición 2.5 (MQA y GQA). Sea H divisible por G. La *atención de consultas agrupadas* con G grupos (GQA-G) divide las H cabezas de consulta en G grupos de H/G cabezas consecutivas; todas las cabezas de un grupo comparten una única cabeza de clave y una de valor [1, §2.2]. Con proyecciones $W_Q^{(i)} \in \mathbb{R}^{d\times d_h}$ para $i = 1,\dots,H$ y $W_K^{(g)}, W_V^{(g)} \in \mathbb{R}^{d\times d_h}$ para $g = 1,\dots,G$, la cabeza i usa la clave y el valor del grupo $g(i) = \lceil iG/H \rceil$. El caso $G = 1$ es la *atención multiconsulta* (MQA) [24, §3], y el caso $G = H$ es la atención multicabeza.

Proposición 2.6 (Ahorro de GQA). *Con $n_{\text{kv}} = G$, respecto de MHA:*

(i) la caché KV se divide exactamente por H/G;

(ii) las proyecciones de clave y valor pasan de $2d^2$ a $2d\,Gd_h$ parámetros, y las de consulta y salida no cambian;

(iii) la intensidad de la proposición 2.4 se multiplica por H/G.

Demostración. (i) y (iii) se siguen de sustituir $n_{\text{kv}} = G$ en las proposiciones 2.2 y 2.4. (ii) Con MHA hay H proyecciones $d \times d_h$ para K y otras tantas para V, es decir, $2Hdd_h = 2d^2$

parámetros si $Hd_h = d$; con GQA hay G de cada tipo, $2Gdd_h$. Las consultas siguen teniendo H cabezas y W_O sigue siendo $d \times d$. ■

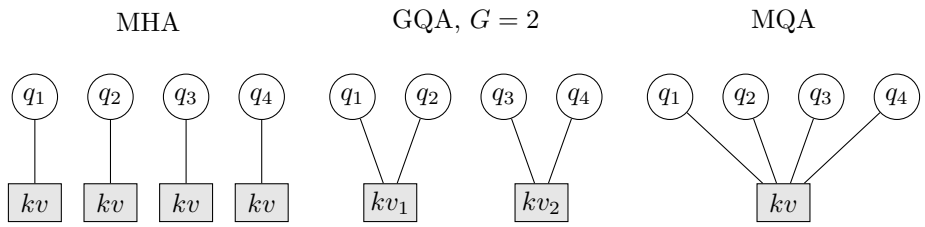


Figura 2.1: Cabezas de consulta y cabezas de clave y valor en MHA, GQA con dos grupos y MQA, con $H = 4$.

Shazeer observó que MQA reduce el tiempo de decodificación por token de 46 a 3,8 microsegundos en su experimento de traducción, con una pérdida de calidad pequeña pero apreciable [24, §4.2 y §4.3]. Ainslie *et al.* proponen GQA como término medio y muestran, además, que un modelo entrenado con MHA puede convertirse en uno con GQA sin entrenarlo desde cero: se promedian las proyecciones de clave y valor de las cabezas de cada grupo y se continúa el entrenamiento durante un 5 % de los pasos originales (*uptraining*) [1, §2.1 y §3.1]. GQA con 8 grupos queda cerca de MHA en calidad y cerca de MQA en velocidad. Llama 2 la usa en sus modelos de 34B y 70B [28, tabla 1], y Llama 3, Mixtral y Grok-1 en todos sus tamaños (cuadro 1.2).

Ejemplo 2.7 (Grok-1). Grok-1 tiene $L = 64$, $H = 48$ cabezas de consulta y $n_{\text{kv}} = 8$, con $d_h = 128$ y contexto máximo de 8192 tokens [30]. Por la proposición 2.2, en bf16 la caché de una secuencia completa ocupa $2 \cdot 64 \cdot 8 \cdot 128 \cdot 8192 \cdot 2 = 2^{31}$ bytes = 2 GiB; con MHA serían $48/8 = 6$ veces más, 12 GiB.

2.4 Atención latente multicabeza

GQA reduce la caché eliminando cabezas de clave y valor. MLA, introducida en DeepSeek-V2, conserva H cabezas, pero guarda en la caché un único vector de baja dimensión por token del que se reconstruyen todas las claves y todos los valores [6, §2.1.2].

Definición 2.8 (MLA sin codificación de posición). Sean $d_c \ll Hd_h$, $W^{DKV} \in \mathbb{R}^{d \times d_c}$ y, para cada cabeza i, $W_i^{UK}, W_i^{UV} \in \mathbb{R}^{d_c \times d_h}$ y $W_i^{Q} \in \mathbb{R}^{d \times d_h}$. Para el token t con entrada $h_t \in \mathbb{R}^{1 \times d}$,

$$c_t = h_t W^{DKV} \in \mathbb{R}^{1 \times d_c}, \qquad k_{t,i} = c_t W_i^{UK}, \qquad v_{t,i} = c_t W_i^{UV}, \qquad q_{t,i} = h_t W_i^{Q}.$$

La caché guarda solo los vectores latentes $c_1, \ldots, c_T$.

Guardar c_t en lugar de $k_{t,i}$ y $v_{t,i}$ solo ahorra memoria si no hay que reconstruir explícitamente las claves y los valores en cada paso. La factorización lo evita.

Proposición 2.9 (Absorción de las proyecciones). *Con la notación de la definición 2.8, sean $\tilde{q}_{t,i} = q_{t,i}(W_i^{UK})^\top \in \mathbb{R}^{1 \times d_c}$ y $a_{t,i,j}$ los pesos de atención de la cabeza i. Entonces*

$$q_{t,i}\, k_{j,i}^\top = \tilde{q}_{t,i}\, c_j^\top, \qquad \sum_{j \le t} a_{t,i,j}\, v_{j,i} = \Big(\sum_{j \le t} a_{t,i,j}\, c_j\Big) W_i^{UV}.$$

En consecuencia, las puntuaciones y la salida de cada cabeza se calculan a partir de los c_j, sin formar $k_{j,i}$ ni $v_{j,i}$; además, W_i^{UK} puede integrarse en la proyección de consulta y W_i^{UV}, en la de salida.

Demostración. $q_{t,i}k_{j,i}^\top = q_{t,i}(c_j W_i^{UK})^\top = q_{t,i}(W_i^{UK})^\top c_j^\top = \tilde{q}_{t,i}c_j^\top$, por la regla de la traspuesta de un producto. Para la segunda igualdad, $\sum_j a_{t,i,j}c_j W_i^{UV} = (\sum_j a_{t,i,j}c_j)W_i^{UV}$ por la distributividad del producto de matrices. Por último, $\tilde{q}_{t,i} = h_t\big(W_i^Q(W_i^{UK})^\top\big)$, con $W_i^Q(W_i^{UK})^\top \in \mathbb{R}^{d\times d_c}$ fija, y la salida de la cabeza se multiplica después por el bloque i de W_O, de modo que $W_i^{UV}W_O^{(i)}$ es también una matriz fija. ∎

Observación 2.10 (RoPE desacoplada). La absorción falla con RoPE. Con vectores fila, la rotación de la posición m actúa como $x \mapsto xR_m$, y $R_m R_n^\top = R_{m-n}$ (es la versión traspuesta de la identidad del módulo 14, tema 4). Si la consulta y la clave se rotan, la puntuación es $q_{t,i}R_t(c_j W_i^{UK}R_j)^\top = q_{t,i}R_{t-j}(W_i^{UK})^\top c_j^\top$: entre $q_{t,i}$ y c_j aparece una matriz que depende de la distancia $t-j$, y ya no hay una matriz fija que integrar en la consulta. DeepSeek-V2 lo resuelve añadiendo a cada cabeza una componente de dimensión d_h^R que sí se rota y cuya clave k_t^R es compartida por todas las cabezas y se guarda también en la caché [6, §2.1.3]. La caché por token y capa tiene entonces $d_c + d_h^R$ números.

Cuadro 2.1: Números guardados en la caché KV por token, en un modelo de L capas [6, tabla 1].

Mecanismo	Caché por token	DeepSeek-V2 ($L=60$, $H=d_h=128$)
MHA	$2Hd_hL$	1 966 080
GQA-G	$2Gd_hL$	15 360 G
MQA	$2d_hL$	15 360
MLA	$(d_c+d_h^R)L$	$(512+64)\cdot 60 = 34\,560$

Ejemplo 2.11 (DeepSeek-V2). DeepSeek-V2 usa $L=60$, $H=128$, $d_h=128$, $d_c=512=4d_h$ y $d_h^R=64=d_h/2$ [6]. La caché por token tiene $(d_c+d_h^R)L=\frac{9}{2}d_hL=34\,560$ números, frente a 1 966 080 con MHA: 56,89 veces menos. Equivale a la caché de GQA con $576/(2\cdot128)=2{,}25$ grupos. En bf16 son 69 120 bytes = 67,5 KiB por token; un contexto de $2^{17}=131\,072$ tokens ocupa 8,44 GiB, frente a 480 GiB con MHA. Según el artículo, MLA supera a MHA en calidad en sus experimentos y reduce la caché un 93,3 % respecto de su modelo denso anterior [6].

Lecturas recomendadas para este tema

- Shazeer [24], §2.4 «Multihead Attention (Incremental)» y §3 «Multi-Query Attention».
- Ainslie *et al.* [1], §2.1 «Uptraining» y §2.2 «Grouped-query attention».
- DeepSeek-AI [6], §2.1 «Multi-Head Latent Attention», con la tabla 1.
- Stanford CS336 [16], clase 10, «Inference»: reducción de la caché KV (GQA, MLA, atención entre capas y atención local).
- Temario de CMU 11-868 [5]: sistemas de inferencia y servicio de modelos.

Ejercicios del tema 2

Ejercicio 2.1 ⋆ Justifique si son verdaderas o falsas: (a) GQA reduce los FLOPs de la proyección de consulta; (b) con MQA, la caché KV por token y capa no depende del número de cabezas de consulta; (c) MLA guarda en la caché las claves y los valores de cada cabeza en una dimensión reducida.

Ejercicio 2.2 ⋆⋆ Un modelo tiene 32 capas, 32 cabezas de consulta con $d_h = 128$ y trabaja en bf16. Calcule la caché KV de una secuencia de $T = 8192$ tokens con MHA, con GQA-8 y con MQA, en las unidades binarias más adecuadas.

Ejercicio 2.3 ⋆⋆ Sea un modelo con la forma de Llama 2 13B: $V = 32\,000$, $d = 5120$, $d_{\text{ff}} = 13\,824$, $L = 40$ y 40 cabezas de consulta con $d_h = 128$. Se sirve en bf16 con secuencias de $T = 1024$ tokens, en un acelerador con ancho de banda de memoria de 3,35 TB/s, y se supone que cada paso de decodificación lee una vez todos los pesos y toda la caché. Para (a) MHA ($n_{\text{kv}} = 40$) y (b) GQA con $n_{\text{kv}} = 8$, ambos con lote $b = 64$, calcule los parámetros, la memoria total, la latencia por paso y el rendimiento en tokens por segundo.

Fuente: adaptado de Stanford CS336 [16], clase 10 («Inference»), ejemplo de Llama 2 13B con caché reducida mediante GQA.

Ejercicio 2.4 ⋆⋆ Con la proposición 2.4, calcule la intensidad aritmética de la atención en la decodificación de Llama 2 70B en bf16, con MHA, con su GQA-8 y con MQA. Compare cada valor con $I^* \approx 156$ FLOP/byte y calcule qué fracción del rendimiento máximo de la A100 se alcanza según el modelo *roofline*.

Ejercicio 2.5 ⋆⋆ Un modelo hipotético con MLA tiene $L = 40$, $H = 32$, $d_h = 128$, $d_c = 256$ y $d_h^R = 64$. (a) Calcule la caché por token con MLA y con MHA, en número de valores y en bytes (bf16). (b) ¿A cuántos grupos de GQA equivale? (c) ¿Cuántos tokens de contexto caben en 16 GiB de caché con cada mecanismo?

Ejercicio 2.6 ⋆⋆⋆ Sea un modelo MHA en el que, dentro de cada grupo de H/G cabezas, todas las proyecciones de clave coinciden entre sí y lo mismo ocurre con las de valor. (a) Demuestre que su conversión a GQA-G promediando las proyecciones de cada grupo produce exactamente las mismas salidas. (b) Muestre con un ejemplo de dos cabezas con $d_h = 1$ que, si las proyecciones no coinciden, las puntuaciones cambian en general.

Ejercicio 2.7 ⋆⋆⋆ Con la notación de la definición 2.8 y vectores fila, suponga que la clave se rota, $k_{j,i} = c_j W_i^{UK} R_j$, y la consulta, $q_{t,i} R_t$, donde R_m es la matriz ortogonal de RoPE en la posición m y $R_m R_n^\top = R_{m-n}$. (a) Escriba la puntuación $q_{t,i} R_t (c_j W_i^{UK} R_j)^\top$ como $q_{t,i}\, M_{t,j}\, c_j^\top$ e identifique $M_{t,j}$. (b) Suponiendo que todas las frecuencias de RoPE cumplen $0 < \theta_l < 2\pi$, demuestre que $M_{t,j}$ toma el mismo valor para todos los pares (t, j) solo si $W_i^{UK} = 0$, y explique por qué esto impide la absorción.

Tema 3

Atención local y contexto largo

La atención causal completa evalúa del orden de $T^2/2$ pares de posiciones por capa, y su caché crece linealmente con T. Este tema estudia dos respuestas complementarias. La primera limita a cada token a una ventana de posiciones recientes, de modo que el coste por capa pasa a ser lineal en T y la caché queda acotada; la profundidad del modelo recupera parte del alcance perdido. La segunda permite que un modelo entrenado con un contexto corto funcione con uno más largo, modificando las frecuencias de RoPE. Se sigue a Jiang *et al.* [19, §2] para la ventana deslizante, a Beltagy *et al.* [2, §3.1] para los patrones dispersos, a Chen *et al.* [3, §2] y a Peng *et al.* [22, §2–3] para la extensión del contexto, y las clases 3 y 10 de CS336 [16].

3.1 Atención con ventana deslizante

Definición 3.1 (Atención con ventana deslizante). Sea $W \geq 1$. La *atención con ventana deslizante* de tamaño W usa la máscara

$$M_{ij} = \begin{cases} 0 & \text{si } i - W < j \leq i, \\ -\infty & \text{en otro caso,} \end{cases}$$

de modo que la posición i atiende a las $\min(i, W)$ posiciones $j \in \{\max(1, i - W + 1), \ldots, i\}$, incluida ella misma. Con $W \geq T$ se recupera la máscara causal.

Proposición 3.2 (Pares evaluados). *Para una secuencia de T tokens, la atención causal completa evalúa $T(T+1)/2$ pares (i, j) por capa y cabeza. Con ventana $W \leq T$ evalúa*

$$\frac{W(W+1)}{2} + (T - W)\, W = TW - \frac{W(W-1)}{2}$$

pares. Para $T \gg W$, el cociente entre ambos recuentos es aproximadamente $T/(2W)$.

Demostración. La fila i de la atención completa tiene i posiciones permitidas, y $\sum_{i=1}^{T} i = T(T+1)/2$. Con ventana, las filas $i \leq W$ tienen i posiciones, que suman $W(W+1)/2$, y las $T - W$ filas restantes tienen W cada una. Desarrollando, $W(W+1)/2 + TW - W^2 = TW - W(W-1)/2$. El cociente es $\frac{T(T+1)/2}{TW - W(W-1)/2}$; dividiendo numerador y denominador por TW y haciendo $T \to \infty$ con W fijo, tiende a $\frac{T+1}{2W} \approx T/(2W)$. ∎

3.2 Campo receptivo

Una capa con ventana solo mezcla información de W posiciones, pero la capa siguiente recibe vectores que ya resumen otras W, como en una red convolucional.

Teorema 3.3 (Campo receptivo de la atención con ventana). *En una pila de L capas de atención con ventana W, y con conexiones residuales y subcapas posición a posición entre ellas, la salida de la posición i tras la capa L solo puede depender de las entradas en las posiciones $\{i-L(W-1),\dots,i\}$. El campo receptivo tiene, por tanto, $L(W-1)+1$ posiciones.*

Demostración. Por inducción sobre L. Para $L=1$, la posición i lee las posiciones $j\in[i-W+1,i]$, un intervalo de $W=1\cdot(W-1)+1$ posiciones; la conexión residual y el MLP actúan sobre la propia posición i, que ya está incluida. Supóngase el resultado cierto para L capas. La capa $L+1$ en la posición i combina las salidas de la capa L en las posiciones $j\in[i-W+1,i]$, y por la hipótesis cada una depende de las entradas en $[j-L(W-1),j]$. La unión de esos intervalos es $[i-W+1-L(W-1),\,i]=[i-(L+1)(W-1),\,i]$, porque los intervalos son consecutivos y se solapan. Tiene $(L+1)(W-1)+1$ posiciones. ∎

Ejemplo 3.4 (Mistral 7B). Mistral 7B usa $W=4096$ y $L=32$ [19, tabla 1]. Por el teorema 3.3, el campo receptivo de la última capa es $32\cdot4095+1=131\,041$ tokens. El artículo da la aproximación $LW=131\,072$, «unos 131 000 tokens» [19, §2].

Observación 3.5. El teorema 3.3 da una cota superior de la dependencia, no una garantía de que la información llegue. Un dato situado a 10^5 posiciones de distancia solo puede alcanzar la posición i tras pasar por decenas de capas intermedias, cada una de las cuales lo mezcla con otros. Por eso la clase 10 de CS336 advierte que la atención local puede perjudicar la calidad y que la solución habitual es intercalar capas locales y globales [16].

3.3 La caché circular

Definición 3.6 (Caché circular). Con ventana W, la *caché circular* de una capa tiene W ranuras $0,\dots,W-1$. Las claves y los valores de la posición $i\geq0$ se escriben en la ranura $i \bmod W$, sobrescribiendo lo que hubiera [19, §2].

Proposición 3.7 (Corrección de la caché circular). *Tras escribir la posición i, la caché contiene exactamente las claves y los valores de las posiciones $\max(0,i-W+1),\dots,i$, que son las que la posición i necesita.*

Demostración. Los W enteros consecutivos $i-W+1,\dots,i$ tienen restos distintos módulo W: si dos de ellos, $a<b$, tuvieran el mismo resto, W dividiría a $b-a\in\{1,\dots,W-1\}$, lo que es imposible. Ocupan, por tanto, W ranuras distintas, es decir, todas. Cada ranura contiene la última posición escrita en ella, y la última posición $\leq i$ con resto r es la única del intervalo $[i-W+1,i]$ con ese resto. Si $i<W-1$, solo se han escrito las ranuras $0,\dots,i$ y contienen las posiciones $0,\dots,i$. ∎

La memoria de la caché pasa a ser $2Ln_{\text{kv}}d_hWs$ bytes por secuencia, independiente de T. En Mistral 7B ($L=32$, $n_{\text{kv}}=8$, $d_h=128$), en bf16 son 512 MiB, frente a 4 GiB para una secuencia de 32 768 tokens sin ventana: el artículo cita esa reducción de 8× [19, §2].

3.4 Patrones dispersos e híbridos

Longformer combina tres patrones [2, §3.1]: la ventana deslizante; la *ventana dilatada*, en la que cada posición atiende a w posiciones separadas por saltos de tamaño δ, con un campo receptivo del orden de $L\,\delta\,w$; y la *atención global* de unas pocas posiciones elegidas, que

atienden a toda la secuencia y a las que atiende toda la secuencia. El coste sigue siendo $O(Tw)$.

Los modelos recientes prefieren alternar capas. La clase 3 de CS336 cita un modelo con una capa de atención completa cada cuatro y el resto con ventana, y la clase 10 recoge un diseño con una capa global cada seis [16].

Proposición 3.8 (Caché de un modelo híbrido). *Sea un modelo de L capas en el que una fracción ρ usa atención completa y el resto ventana W. Con $T \geq W$, su caché es la de un modelo de atención completa multiplicada por $\rho + (1-\rho)W/T$.*

Demostración. Por la proposición 2.2, la caché es proporcional a la suma, sobre las capas, del número de posiciones guardadas: $\rho L \cdot T + (1-\rho)L \cdot W$. Dividiendo por LT se obtiene el factor. ∎

Con $\rho = 1/4$, $W = 4096$ y $T = 131\,072$, el factor es $0{,}25 + 0{,}75/32 = 0{,}2734$: la caché se reduce a algo más de una cuarta parte, dominada por las capas globales.

3.5 RoPE y longitudes de onda

RoPE (módulo 14, tema 4) gira cada par de coordenadas $i \in \{0, \ldots, d_h/2 - 1\}$ de consultas y claves un ángulo $m\theta_i$ en la posición m, con $\theta_i = \beta_0^{-2i/d_h}$ y base $\beta_0 = 10^4$ en el artículo original [27, §3.2.2]. Llama 3 usa $\beta_0 = 5 \cdot 10^5$ [13, tabla 3].

Definición 3.9 (Longitud de onda). La *longitud de onda* del par i es $\lambda_i = 2\pi/\theta_i = 2\pi\beta_0^{2i/d_h}$: el número de posiciones tras el cual la rotación da una vuelta completa [22, ec. 8].

Las longitudes de onda crecen geométricamente desde $\lambda_0 = 2\pi \approx 6{,}283$ hasta $\lambda_{d_h/2-1} = 2\pi\beta_0^{1-2/d_h}$. Con $d_h = 128$ y $\beta_0 = 10^4$, la mayor es 54 410; con $\beta_0 = 5 \cdot 10^5$, unos $2{,}56 \cdot 10^6$. Los pares de longitud de onda corta codifican la posición relativa con precisión; los de longitud de onda mayor que el contexto de entrenamiento no completan nunca una vuelta durante el entrenamiento, de modo que se comportan casi como una codificación absoluta de la posición.

3.6 Extensión del contexto

Sea un modelo entrenado con contexto L_0 que se quiere usar con $L' > L_0$, y sea $s_e = L'/L_0$. Usar posiciones $m \geq L_0$ sin más (extrapolación) produce ángulos $m\theta_i$ que los pares de onda larga nunca vieron, y la calidad se degrada rápidamente [3, §2.2].

Definición 3.10 (Interpolación de posiciones). La *interpolación de posiciones* (PI) sustituye la posición m por m/s_e antes de aplicar RoPE: el ángulo del par i pasa a ser $m\theta_i/s_e$ [3, §2.3, ec. 4].

Con PI, todo ángulo usado con $m < L'$ es menor que $L_0\theta_i$, es decir, está dentro del intervalo visto en el entrenamiento. Chen *et al.* demuestran que el error al interpolar entre posiciones conocidas está acotado por una cantidad mucho menor que la cota correspondiente a la extrapolación [3, §2.3, teorema 2.1], y extienden modelos LLaMA hasta 32 768 tokens con unos 1000 pasos de ajuste. El inconveniente es que PI comprime también los pares de onda

corta, que no lo necesitaban, y la resolución entre posiciones cercanas empeora en un factor s_{e}.

Definición 3.11 (Cambio de base NTK). La interpolación *consciente del NTK* conserva las posiciones y cambia la base por

$$\beta_0' = \beta_0 \, s_{\mathrm{e}}^{d_h/(d_h-2)}$$

[22, §3.1].

Proposición 3.12 (Efecto del cambio de base). *Con la base β_0' de la definición 3.11, las nuevas frecuencias son $\theta_i' = \theta_i \, s_{\mathrm{e}}^{-2i/(d_h-2)}$. En particular, $\theta_0' = \theta_0$ (el par más rápido no cambia) y $\theta_{d_h/2-1}' = \theta_{d_h/2-1}/s_{\mathrm{e}}$ (el más lento se interpola exactamente como en PI). Las frecuencias intermedias se dividen por factores que crecen geométricamente con i entre 1 y s_{e}.*

Demostración. $\theta_i' = (\beta_0')^{-2i/d_h} = \beta_0^{-2i/d_h} \, s_{\mathrm{e}}^{-\frac{d_h}{d_h-2}\cdot\frac{2i}{d_h}} = \theta_i \, s_{\mathrm{e}}^{-2i/(d_h-2)}$. Con $i = 0$ el exponente es 0; con $i = d_h/2 - 1$ es $-2(d_h/2-1)/(d_h-2) = -1$. El factor $s_{\mathrm{e}}^{2i/(d_h-2)}$ es una progresión geométrica en i de razón $s_{\mathrm{e}}^{2/(d_h-2)} > 1$. ∎

YaRN refina esta idea en dos pasos [22, §3.2 y §3.3]. Primero, decide para cada par según el cociente $r_i = L_0/\lambda_i$, que cuenta cuántas vueltas completa en el contexto original: si $r_i > \beta$ (onda corta) no interpola; si $r_i < \alpha$ (onda larga) interpola como PI; entre ambos valores mezcla linealmente las dos opciones. Para Llama, $\alpha = 1$ y $\beta = 32$. Segundo, multiplica los logits de la atención por un factor de temperatura, implementado escalando consultas y claves por $\sqrt{1/t} = 0{,}1 \ln s_{\mathrm{e}} + 1$ [22, §3.3]. Con $s_{\mathrm{e}} = 16$, $\sqrt{1/t} = 1{,}277$. Los autores extienden Llama 2 a 128 000 tokens con 400 pasos de ajuste [22, §4.1].

Ejemplo 3.13 (Qué pares interpola YaRN). Con $L_0 = 4096$, $d_h = 128$ y $\beta_0 = 10^4$, $r_i = 4096/(2\pi \cdot 10^{4 \cdot 2i/128})$. Para $i = 0$, $r_0 = 651{,}9$; para $i = 20$, $r_{20} = 36{,}66$; para $i = 40$, $r_{40} = 2{,}061$; para $i = 46$, $r_{46} = 0{,}8693$. Con $\alpha = 1$ y $\beta = 32$, los pares $i = 0, \ldots, 20$ no se interpolan, los pares $i = 46, \ldots, 63$ se interpolan como en PI y los 25 intermedios se mezclan.

Observación 3.14 (Entrenamiento por etapas). La extensión del contexto también puede hacerse durante el preentrenamiento. Llama 3 405B se entrena primero con 8192 tokens y amplía el contexto hasta 128 000 en seis etapas, con unos $8 \cdot 10^{11}$ tokens, al final del preentrenamiento, porque el coste de la atención crece cuadráticamente con la longitud [13, §3.4.2]. El informe de Gemini 1.5 afirma que el modelo recupera información en contextos de hasta 10^7 tokens, pero no publica el mecanismo que lo permite [12, §3.1].

Lecturas recomendadas para este tema

- Jiang *et al.* [19], §2: «Sliding Window Attention» y «Rolling Buffer Cache».
- Beltagy *et al.* [2], §3.1 «Attention Pattern».
- Chen *et al.* [3], §2.3 «Proposed approach: Position Interpolation».
- Peng *et al.* [22], §2 y §3 (NTK-aware, NTK-by-parts y YaRN).
- Su *et al.* [27], §3.3 y §3.4.3 «Long-term decay of RoPE».
- Zhang *et al.* [32], §11.6.2 «Comparing CNNs, RNNs, and Self-Attention».

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Justifique si son verdaderas o falsas: (a) con ventana deslizante, la caché KV de una capa no depende de la longitud de la secuencia; (b) la interpolación de posiciones comprime por igual todos los pares de frecuencias, y el cambio de base NTK no; (c) en una sola capa con ventana W, la posición i puede recibir información de la posición $i - W$.

Ejercicio 3.2 ⋆⋆ Con $T = 16\,384$ y $W = 4096$, cuente exactamente los pares que evalúan la atención causal completa y la de ventana, calcule su cociente y compárelo con la aproximación $T/(2W)$.

Ejercicio 3.3 ⋆⋆ Una caché circular tiene $W = 4$ ranuras y se procesan las posiciones $0, 1, \ldots, 9$. (a) ¿En qué ranura se escribe cada posición? (b) ¿Qué posición contiene cada ranura tras procesar la posición 9? (c) Compruebe la proposición 3.7 para $i = 9$.

Ejercicio 3.4 ⋆⋆ Sean $d_h = 128$, $\beta_0 = 10^4$ y $s_e = 4$. (a) Calcule la nueva base de la definición 3.11. (b) Calcule θ_0, θ_{32} y θ_{63} originales, con PI y con el cambio de base NTK. (c) ¿Por qué factor divide cada método la frecuencia θ_{32}? (d) Calcule el factor de YaRN $\sqrt{1/t}$.

Ejercicio 3.5 ⋆ Una capa de autoatención sobre n tokens de dimensión d cuesta $O(n^2 d)$ operaciones; una capa recurrente con estado de dimensión d, $O(nd^2)$; una convolución con núcleo de tamaño k, $O(knd^2)$. Con $n = 4096$, $d = 1024$ y $k = 3$, evalúe las tres expresiones. ¿A partir de qué n la atención es más cara que la capa recurrente? ¿Qué limitación tiene la convolución que no tiene la atención?

Fuente: adaptado de Zhang *et al.* [32], §11.6.2 («Comparing CNNs, RNNs, and Self-Attention») y §11.6.5, ejercicio 1.

Ejercicio 3.6 ⋆⋆⋆ Una ventana dilatada de tamaño w y dilatación δ permite que la posición i atienda a las posiciones i, $i-\delta$, $i-2\delta, \ldots, i-(w-1)\delta$. Demuestre que, tras L capas, la salida en i solo depende de entradas en $[i - L\delta(w-1),\, i]$, y que la posición $i - L\delta(w-1)$ sí puede influir. ¿Influyen todas las posiciones del intervalo? Relacione el resultado con la estimación $L\delta w$ de Longformer.

Ejercicio 3.7 ⋆⋆⋆ (a) Demuestre que el cambio de base NTK es la única elección de base β_0' que deja θ_0 fija y divide $\theta_{d_h/2-1}$ exactamente por s_e. (b) Demuestre que, con PI, el mayor ángulo usado por el par i en un contexto de L' tokens es el mismo que en el entrenamiento con L_0 tokens.

Tema 4

Mezcla de expertos

Un Transformer denso aplica a cada token todos sus parámetros. Una capa de *mezcla de expertos* (MoE) sustituye el MLP por varios MLP alternativos y por un enrutador que elige unos pocos para cada token, de modo que los parámetros crecen con el número de expertos y el cómputo por token solo con el número de expertos elegidos. Este tema formula la capa, distingue parámetros totales y activos, estudia el problema del balanceo de carga y sus soluciones, y reconstruye los recuentos publicados de varios modelos. Se sigue la clase 4 de CS336 [16], los artículos de Shazeer *et al.* [26, §2 y §4], Fedus *et al.* [10, §2] y Jiang *et al.* [18, §2], y los informes de DeepSeek-V2 y DeepSeek-V3 [6, §2.2], [7, §2.1.2].

4.1 La capa de mezcla de expertos

Definición 4.1 (Capa MoE con enrutado top-k). Sean $E_1, \dots, E_N : \mathbb{R}^{1\times d} \to \mathbb{R}^{1\times d}$ MLP con la misma forma (los *expertos*), $W_r \in \mathbb{R}^{d\times N}$ la matriz del *enrutador* y $1 \le k \le N$. Para un token x, sean $z = xW_r \in \mathbb{R}^N$ sus logits y $\mathcal{T}(x)$ el conjunto de los índices de los k logits mayores. La salida de la capa es

$$y = \sum_{i\in\mathcal{T}(x)} g_i(x)\, E_i(x), \qquad g_i(x) = \frac{e^{z_i}}{\sum_{j\in\mathcal{T}(x)} e^{z_j}}.$$

Los expertos que no están en $\mathcal{T}(x)$ no se evalúan. Esta es la formulación de Mixtral, $G(x) = \mathrm{softmax}(\mathrm{TopK}(xW_r))$ [18, §2.1].

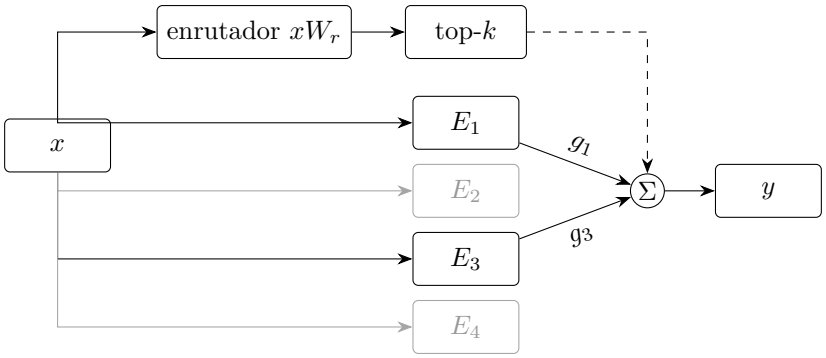


Figura 4.1: Capa MoE con $N = 4$ y $k = 2$: el enrutador elige los expertos 1 y 3, que son los únicos que se evalúan, y la salida es su combinación con pesos $g_1 + g_3 = 1$.

Hay variantes en el orden de las operaciones. Shazeer *et al.* también normalizan solo entre los elegidos, pero añaden ruido gaussiano a los logits, con una desviación aprendida, $H(x)_i = (xW_r)_i + \xi_i \,\mathrm{softplus}\big((xW_{\text{ruido}})_i\big)$ con $\xi_i \sim \mathcal{N}(0,1)$, para favorecer la exploración y el balanceo [26, §2.1]. El Switch Transformer usa $k = 1$ y pondera la salida por la probabilidad

de la softmax sobre los N expertos, sin renormalizar [10, §2.1]. La clase 4 de CS336 recoge que DeepSeek-V1 y V2, Grok y Qwen aplican la softmax antes del top-k, y que Mixtral, DBRX y DeepSeek-V3 normalizan después [16].

Proposición 4.2 (Renormalización). *Sean $p = \mathrm{softmax}(z)$ las probabilidades sobre los N expertos y g los pesos de la definición 4.1. Para $i \in \mathcal{T}(x)$, $g_i = p_i / \sum_{j\in\mathcal{T}(x)} p_j \geq p_i$, con igualdad para todo i si y solo si $k = N$. Además, el orden de los expertos elegidos es el mismo con g que con p.*

Demostración. Dividiendo numerador y denominador de g_i por $Z = \sum_{j=1}^{N} e^{z_j}$, $g_i = (e^{z_i}/Z)/\sum_{j\in\mathcal{T}}(e^{z_j}/Z) = p_i/\sum_{j\in\mathcal{T}} p_j$. Como $0 < \sum_{j\in\mathcal{T}} p_j \leq 1$, $g_i \geq p_i$, y la suma vale 1 si y solo si $\mathcal{T}$ contiene todos los índices, porque cada $p_j > 0$. El orden se conserva porque g es p multiplicada por una constante positiva. ■

Ejemplo 4.3 (Enrutador con cuatro expertos). Sean $z = (2;\ 1;\ 0{,}5;\ -1)$ y $k = 2$. Los elegidos son los expertos 1 y 2, con $g_1 = e^2/(e^2 + e^1) = 1/(1 + e^{-1}) = 0{,}7311$ y $g_2 = 0{,}2689$, de modo que $y = 0{,}7311\, E_1(x) + 0{,}2689\, E_2(x)$. Con la softmax sobre los cuatro, $p_1 = 0{,}6095$ y $p_2 = 0{,}2242$, que suman 0,8337; dividiendo por esa suma se recuperan g_1 y g_2, como afirma la proposición 4.2.

4.2 Parámetros totales y activos

Proposición 4.4 (Parámetros totales y activos). *Sea un modelo con L capas MoE de N expertos de P_E parámetros cada uno, k de ellos activos por token, y sea P_0 el número de parámetros compartidos por todos los tokens (atención,* embeddings*, normalizaciones y enrutadores). Entonces*

$$N_{\mathrm{tot}} = P_0 + LNP_E, \qquad N_{\mathrm{act}} = P_0 + LkP_E.$$

El cómputo por token en la pasada hacia delante es de unos $2N_{\mathrm{act}}$ FLOPs y el de entrenamiento con D tokens, $C \approx 6N_{\mathrm{act}}D$, mientras que la memoria de los pesos es proporcional a N_{tot}.

Demostración. Cada token usa todos los parámetros compartidos y, en cada una de las L capas, los de k expertos. Los FLOPs se siguen de la regla de dos operaciones por parámetro de matriz y token en la pasada hacia delante y seis en total con la retropropagación (módulos 14 y 16), aplicada a los parámetros que intervienen en el cálculo de ese token. Para la memoria, como el enrutador puede elegir cualquier experto en cualquier token, todos deben estar cargados. ■

Ejemplo 4.5 (Mixtral 8x7B). Mixtral tiene $d = 4096$, $d_{\mathrm{ff}} = 14\,336$ (SwiGLU, tres matrices), $L = 32$, un vocabulario de 32 000 tokens, 32 cabezas de consulta y 8 de clave y valor de dimensión 128, y $N = 8$ expertos con $k = 2$ [18, tabla 1]. Con *embeddings* de entrada y salida separados y RMSNorm con un vector de ganancias por norma:

1. un experto: $P_E = 3 \cdot 4096 \cdot 14\,336 = 176\,160\,768$; los $8 \cdot 32$ expertos suman 45 097 156 608;
2. atención por capa: $2 \cdot 4096^2 + 2 \cdot 4096 \cdot 1024 = 41\,943\,040$; en 32 capas, 1 342 177 280;
3. *embeddings* y salida: $2 \cdot 32\,000 \cdot 4096 = 262\,144\,000$; enrutadores: $32 \cdot 4096 \cdot 8 = 1\,048\,576$; normas: $65 \cdot 4096 = 266\,240$;
4. $N_{\mathrm{tot}} = 46\,702\,792\,704$ y $N_{\mathrm{act}} = 12\,879\,925\,248$, un 27,58 %.

El artículo da 47 000 y 13 000 millones [18]. El nombre «8x7B» no significa $56 \cdot 10^9$ parámetros: la atención y los *embeddings* se comparten.

4.3 Balanceo de carga

Si un experto empieza siendo algo mejor, el enrutador le envía más tokens, recibe más gradiente, mejora más y acaba acaparando el tráfico; los demás apenas se entrenan. Shazeer *et al.* ya observaron este colapso y lo combatieron con una pérdida sobre el coeficiente de variación de la importancia de cada experto, $\text{Importance}(X) = \sum_{x \in X} G(x)$ [26, §4]. La formulación más usada es la del Switch Transformer.

Definición 4.6 (Pérdida de balanceo). Sea un lote $\mathcal{B}$ de tokens. Para cada experto i, sean f_i la fracción de las asignaciones del lote que recibe y $P_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} p_i(x)$ la probabilidad media que le da el enrutador. La *pérdida de balanceo* es

$$\mathcal{L}_{\text{bal}} = N \sum_{i=1}^{N} f_i P_i,$$

y se suma a la entropía cruzada con un coeficiente α; el Switch Transformer usa $\alpha = 10^{-2}$ [10, §2.2, ecs. 4–6].

Las fracciones f_i proceden de una selección discreta y no son derivables; el gradiente fluye solo a través de P_i. La clase 4 de CS336 lo resume así: la derivada respecto de $p_i(x)$ es proporcional al número de tokens asignados al experto i, de modo que un experto más usado ve reducida su probabilidad con más fuerza [16].

Teorema 4.7 (Cotas de la pérdida de balanceo). *Si $f = P$ (el tráfico real coincide con la probabilidad media), entonces $1 \le \mathcal{L}_{\text{bal}} \le N$. La cota inferior se alcanza si y solo si $P_i = 1/N$ para todo i, y la superior, si y solo si un experto recibe todo el tráfico.*

Demostración. Con $f = P$, $\mathcal{L}_{\text{bal}} = N \sum_i P_i^2$, con $P_i \ge 0$ y $\sum_i P_i = 1$. Por la desigualdad de Cauchy–Schwarz aplicada a $(1, \dots, 1)$ y $(P_1, \dots, P_N)$,

$$1 = \Big(\sum_{i=1}^{N} 1 \cdot P_i \Big)^2 \le \Big(\sum_{i=1}^{N} 1 \Big) \Big(\sum_{i=1}^{N} P_i^2 \Big) = N \sum_{i=1}^{N} P_i^2,$$

con igualdad si y solo si los dos vectores son proporcionales, es decir, si todos los P_i son iguales a $1/N$. Para la cota superior, $0 \le P_i \le 1$ implica $P_i^2 \le P_i$, luego $\sum_i P_i^2 \le \sum_i P_i = 1$ y $\mathcal{L}_{\text{bal}} \le N$; la igualdad exige $P_i^2 = P_i$ para todo i, es decir, $P_i \in \{0, 1\}$, y como suman 1, un único P_i vale 1. ∎

Cuadro 4.1: Valores de $\mathcal{L}_{\text{bal}}$ con $N = 4$.

Situación	f	P	$\mathcal{L}_{\text{bal}}$
Reparto uniforme	$(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$	$(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$	1
Desequilibrio	(0,5; 0,3; 0,1; 0,1)	(0,4; 0,3; 0,2; 0,1)	1,28
Colapso	$(1, 0, 0, 0)$	$(1, 0, 0, 0)$	4

Observación 4.8 (Balanceo sin pérdida auxiliar). DeepSeek-V3 sustituye casi por completo la pérdida auxiliar por un sesgo b_i por experto que solo interviene en la selección: los expertos se eligen según $s_i(x) + b_i$, pero se ponderan con $s_i(x)$, y al final de cada paso b_i disminuye en una cantidad fija γ si el experto ha estado sobrecargado y aumenta en γ si ha estado infrautilizado [7, §2.1.2]. Mantiene, con un coeficiente muy pequeño, una pérdida de balanceo por secuencia. DeepSeek-V3 usa además afinidades sigmoides, $s_i(x) = \sigma(u\, e_i^\top)$, en lugar de una softmax.

4.4 Capacidad y descarte de tokens

En un acelerador, cada experto procesa un tensor de tamaño fijo, de modo que el número de tokens que puede recibir por lote está acotado.

Definición 4.9 (Capacidad de un experto). Con un lote de B_t tokens, N expertos, k asignaciones por token y un *factor de capacidad* $c \geq 1$, la *capacidad* de cada experto es

$$C_\mathrm{exp} = c \cdot \frac{k\, B_\mathrm{t}}{N}$$

tokens [10, §2.2, ec. 3]. Las asignaciones que exceden la capacidad se *descartan*: para ellas, ese experto no se evalúa.

Un token descartado en todos sus expertos sale de la capa con su vector intacto, porque el bloque calcula $h + \mathrm{FFN}(\cdot)$ y la contribución del MLP es nula: no se pierde la información del token, sino la transformación de esa capa. Con $B_\mathrm{t} = 1024$, $N = 8$, $k = 1$ y $c = 1{,}25$, la capacidad es $1{,}25 \cdot 128 = 160$ tokens por experto. Un factor mayor descarta menos tokens, pero reserva más memoria y cómputo para posiciones de relleno. La clase 4 de CS336 señala una consecuencia práctica: como el descarte se decide por lote, las peticiones de otros usuarios pueden provocar el descarte de los tokens propios, lo que añade no determinismo a la inferencia [16].

4.5 Expertos de grano fino y compartidos

DeepSeekMoE divide cada experto en m expertos más pequeños y activa m veces más, de modo que los parámetros y el cómputo no cambian, y reserva algunos expertos *compartidos* que procesan todos los tokens [6, §2.2.1]. DeepSeek-V2 usa 2 expertos compartidos y 160 enrutados, de los que activa 6; DeepSeek-V3, 1 compartido y 256 enrutados, de los que activa 8 [6, 7].

Proposición 4.10 (Combinaciones de expertos). *Si cada uno de N expertos se divide en m expertos de tamaño $1/m$ y se activan mk de ellos, el número de combinaciones posibles pasa de $\binom{N}{k}$ a $\binom{mN}{mk}$, con los mismos parámetros totales y activos.*

Demostración. Los parámetros totales son $mN \cdot P_E/m = N P_E$ y los activos, $mk \cdot P_E/m = k P_E$. El número de subconjuntos de mk elementos de un conjunto de mN es $\binom{mN}{mk}$. ∎

Con $N = 8$, $k = 2$ y $m = 4$, se pasa de $\binom{8}{2} = 28$ combinaciones a $\binom{32}{8} = 10\,518\,300$. La clase 4 de CS336 recoge que las ablaciones de DeepSeek muestran ganancias de los expertos de grano fino y de los compartidos, mientras que las de OLMoE confirman las primeras pero no las segundas [16].

4.6 Casos públicos

Cuadro 4.2: Modelos MoE con información publicada por sus autores.

Modelo	Expertos y enrutado	Parámetros
Shazeer *et al.* (2017) [26]	hasta 131 072 expertos entre capas LSTM, top-k con ruido	hasta $1{,}37 \cdot 10^{11}$ en la capa MoE
Switch-C (2021) [10]	2048 expertos, top-1	$1{,}571 \cdot 10^{12}$ (tabla 9)
Mixtral 8x7B (2024) [18]	8 expertos, top-2	$4{,}7 \cdot 10^{10}$ totales, $1{,}3 \cdot 10^{10}$ activos
Grok-1 (2024) [30]	8 expertos, 2 por token	$3{,}14 \cdot 10^{11}$
DeepSeek-V2 (2024) [6]	2 compartidos y 160 enrutados, 6 activos	$2{,}36 \cdot 10^{11}$ totales, $2{,}1 \cdot 10^{10}$ activos
DeepSeek-V3 (2024) [7]	1 compartido y 256 enrutados, 8 activos	$6{,}71 \cdot 10^{11}$ totales, $3{,}7 \cdot 10^{10}$ activos
Gemini 1.5 Pro (2024) [12]	«Transformer de mezcla de expertos dispersa»	no publicado

Observación 4.11 (Lo que no es público). El informe de Gemini 1.5 describe el modelo como un Transformer MoE disperso, pero no da el número de expertos ni el de parámetros [12, §3.1]. Anthropic no ha publicado la arquitectura de Claude, y xAI no ha publicado la de las versiones de Grok posteriores a Grok-1. Nada de lo que se dice en este tema debe atribuirse a esos modelos.

Observación 4.12 (Paralelismo de expertos). Las MoE añaden una forma de paralelismo a las del módulo 17: cada dispositivo guarda un subconjunto de expertos y los tokens viajan al dispositivo de su experto y vuelven mediante una comunicación de todos con todos (*all-to-all*). DeepSeek-V2 limita el número de dispositivos a los que puede ir cada token para acotar esa comunicación [6, §2.2.2].

Lecturas recomendadas para este tema

- Stanford CS336 [16], clase 4, «Mixture of experts».
- Shazeer *et al.* [26], §2 «The Structure of the Mixture-of-Experts Layer» y §4 «Balancing Expert Utilization».
- Fedus *et al.* [10], §2.1 «Simplifying Sparse Routing» y §2.2 «Efficient Sparse Routing».
- Jiang *et al.* [18], §2.1 «Sparse Mixture of Experts».
- DeepSeek-AI [6], §2.2 «DeepSeekMoE», y [7], §2.1.2.
- Seminario CS25 [11], sesión de N. Tazi sobre el entrenamiento a gran escala y las arquitecturas MoE.

Ejercicios del tema 4

Ejercicio 4.1 ⋆ (a) Explique por qué una MoE con $N = 8$ y $k = 2$ necesita más memoria que un modelo denso con los mismos parámetros activos, y cuantifique el cociente para los parámetros de los MLP. (b) ¿Qué ocurre con un token cuya única asignación llega a un experto lleno? ¿Se pierde su información?

Ejercicio 4.2 ⋆ Justifique si son verdaderas o falsas: (a) el informe técnico de Gemini 1.5 indica que el modelo tiene 8 expertos; (b) en el Switch Transformer cada token se envía a un único experto; (c) DeepSeek-V3 no usa ninguna pérdida de balanceo.

Ejercicio 4.3 ⋆⋆ Logits del enrutador $z = (0{,}5;\ 2;\ 1{,}2;\ 0)$ y $k = 2$. Determine los expertos elegidos y sus pesos con la definición 4.1, y los pesos que tendrían con la softmax sobre los cuatro expertos sin renormalizar.

Ejercicio 4.4 ⋆⋆ (a) Con $N = 4$, $f = (0{,}4;\ 0{,}1;\ 0{,}35;\ 0{,}15)$ y $P = (0{,}35;\ 0{,}15;\ 0{,}35;\ 0{,}15)$, calcule $\mathcal{L}_{\text{bal}}$. (b) Con un lote de 2048 tokens, $N = 16$, $k = 1$ y factor de capacidad 1,0, calcule la capacidad por experto y explique qué ocurre si el reparto no es perfecto.

Ejercicio 4.5 ⋆⋆ El código publicado de Grok-1 fija $d = 6144$, $L = 64$, 48 cabezas de consulta y 8 de clave y valor con $d_h = 128$, 8 expertos con 2 activos, vocabulario de 131 072 tokens, *embeddings* de entrada y salida compartidos y un MLP con tres matrices cuya dimensión intermedia es $d_{\text{ff}} = \lfloor 8d \cdot 2/3 \rfloor$. Reconstruya P_E, N_{tot} y N_{act}, despreciando las normalizaciones, y compare con las cifras del archivo `README`: $3{,}14 \cdot 10^{11}$ parámetros y 2 expertos por token.

Ejercicio 4.6 ⋆⋆⋆ Demuestre que, si f y P son dos distribuciones de probabilidad sobre N expertos cualesquiera (no necesariamente iguales), $0 \le \mathcal{L}_{\text{bal}} \le N$, y que ambas cotas son alcanzables. ¿Por qué el mínimo útil es el del teorema 4.7 y no 0?

Ejercicio 4.7 ⋆⋆⋆ Sea un lote de T tokens con enrutado top-1, $f_i = \frac{1}{T}\sum_x \mathbf{1}[\arg\max_j p_j(x) = i]$ y $P_i = \frac{1}{T}\sum_x p_i(x)$. (a) Tratando f como constante, demuestre que

$$\frac{\partial(\alpha \mathcal{L}_{\text{bal}})}{\partial p_i(x)} = \frac{\alpha N}{T^2} \sum_{x'} \mathbf{1}\big[\arg\max_j p_j(x') = i\big].$$

(b) Con $T = 8$, $N = 4$, $\alpha = 10^{-2}$ y asignaciones $(4, 2, 1, 1)$, evalúe la derivada para cada experto. (c) Explique por qué esta realimentación tiende a equilibrar la carga.

Fuente: adaptado de Stanford CS336 [16], clase 4 («Mixture of experts»), derivada de la pérdida de balanceo del Switch Transformer.

Tema 5

Modelos de espacio de estados

La atención guarda todo el pasado y lo consulta en cada paso; una red recurrente lo resume en un estado de tamaño fijo. Los modelos de espacio de estados (SSM) son redes recurrentes lineales con una estructura heredada de la teoría de sistemas: se definen en tiempo continuo, se discretizan y pueden calcularse como recurrencia, con coste constante por token en la generación, o como convolución, en paralelo durante el entrenamiento. Mamba los hace selectivos, haciendo que sus parámetros dependan de la entrada, y recupera el cálculo paralelo mediante un barrido asociativo. Se sigue a Gu, Goel y Ré [15, §2], a Gu y Dao [14, §2 y §3], la parte de la clase 10 de CS336 dedicada a las alternativas al Transformer [16] y la sesión de A. Gu en el seminario CS25 [11]. Los sistemas discretos y su respuesta al impulso siguen el curso 6.003 del MIT [21].

5.1 Sistemas lineales continuos y su discretización

Definición 5.1 (Modelo de espacio de estados). Sean $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times 1}$ y $C \in \mathbb{R}^{1\times n}$. El *modelo de espacio de estados* continuo con entrada escalar $x(t)$, estado $h(t) \in \mathbb{R}^n$ y salida $y(t)$ es

$$h'(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t)$$

[14, ecs. 1a y 1b]. Un modelo de lenguaje aplica un SSM independiente a cada una de las d componentes de la secuencia de vectores.

Para procesar una secuencia se fija un paso $\Delta > 0$ y se supone que la entrada es constante en cada intervalo de longitud Δ (*retención de orden cero*).

Proposición 5.2 (Discretización por retención de orden cero). *Sea $n = 1$, con $A = a \neq 0$ y $B = \beta$. Si $x(\tau) = x_t$ es constante en $[t\Delta, (t+1)\Delta)$, entonces $h_{t+1} = \bar{A}h_t + \bar{B}x_t$, con*

$$\bar{A} = e^{a\Delta}, \qquad \bar{B} = \frac{e^{a\Delta} - 1}{a}\,\beta.$$

Si $a < 0$, entonces $0 < \bar{A} < 1$. En el caso matricial, $\bar{A} = \exp(\Delta A)$ y $\bar{B} = (\Delta A)^{-1}\big(\exp(\Delta A) - I\big)\Delta B$ [14, ec. 4].

Demostración. Multiplicando la ecuación $h' - ah = \beta x$ por $e^{-a\tau}$ se obtiene $\big(e^{-a\tau}h(\tau)\big)' = e^{-a\tau}\beta x_t$ en el intervalo. Integrando entre $t\Delta$ y $(t+1)\Delta$ y multiplicando por $e^{a(t+1)\Delta}$,

$$h((t+1)\Delta) = e^{a\Delta}h(t\Delta) + \beta x_t \int_0^{\Delta} e^{a(\Delta-u)}\,du = e^{a\Delta}h(t\Delta) + \beta x_t\,\frac{e^{a\Delta}-1}{a}.$$

Si $a < 0$, $a\Delta < 0$ y $0 < e^{a\Delta} < 1$. El caso matricial es idéntico, con la exponencial de una matriz y la integral $\int_0^{\Delta} e^{A(\Delta-u)}du = A^{-1}(e^{\Delta A} - I)$ cuando A es invertible. ∎

S4 usa en su lugar la discretización bilineal, $\bar{A} = (I - \frac{\Delta}{2}A)^{-1}(I + \frac{\Delta}{2}A)$ y $\bar{B} = (I - \frac{\Delta}{2}A)^{-1}\Delta B$ [15, §2.3].

Observación 5.3 (Conexión con la física). Con $a = -1/\tau_c$, la ecuación escalar es la de un circuito RC de constante de tiempo τ_c, y $\bar{A} = e^{-\Delta/\tau_c}$ es la fracción de carga que conserva el condensador tras un intervalo Δ. Un paso pequeño respecto de τ_c da $\bar{A} \approx 1$, una memoria larga; un paso grande da $\bar{A} \approx 0$, y el estado olvida casi todo.

5.2 Recurrencia y convolución

Definición 5.4 (SSM discreto). El SSM discreto con matrices $(\bar{A}, \bar{B}, C)$ y estado inicial $h_0 = 0$ transforma la sucesión $(x_t)_{t\geq 1}$ en

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \qquad y_t = Ch_t$$

[14, ecs. 2a y 2b].

Proposición 5.5 (Forma convolucional). *Si $(\bar{A}, \bar{B}, C)$ no dependen de t, la salida del SSM discreto es la convolución causal*

$$y_t = \sum_{j=0}^{t-1} \bar{K}_j\, x_{t-j}, \qquad \bar{K}_j = C\bar{A}^j\bar{B},$$

y $\bar{K}$ es la respuesta del sistema al impulso unitario ($x_1 = 1$ y $x_t = 0$ para $t > 1$).

Demostración. Se demuestra por inducción que $h_t = \sum_{j=0}^{t-1} \bar{A}^j\bar{B}x_{t-j}$. Para $t = 1$, $h_1 = \bar{A}\cdot 0 + \bar{B}x_1$. Si es cierto para $t-1$, $h_t = \bar{A}\sum_{j=0}^{t-2} \bar{A}^j\bar{B}x_{t-1-j} + \bar{B}x_t = \sum_{j'=1}^{t-1} \bar{A}^{j'}\bar{B}x_{t-j'} + \bar{A}^0\bar{B}x_t$, con $j' = j + 1$. Multiplicando por C se obtiene y_t. Con el impulso unitario, solo sobrevive el término $j = t - 1$ y $y_t = \bar{K}_{t-1}$. ∎

Ejemplo 5.6 (Memoria que se desvanece). Sean $\bar{A} = 0{,}5$, $\bar{B} = C = 1$ y $x = (1, 0, 0, 2)$. Por recurrencia, $h = (1;\ 0{,}5;\ 0{,}25;\ 0{,}125 + 2 = 2{,}125)$. Por convolución, con $\bar{K} = (1;\ 0{,}5;\ 0{,}25;\ 0{,}125)$, $y_4 = \bar{K}_0x_4 + \bar{K}_3x_1 = 2 + 0{,}125 = 2{,}125$. El valor inicial pesa $0{,}5^3 = 0{,}125$ tres pasos después. Estos coeficientes se obtienen, por ejemplo, de $a = -1$ y $\Delta = \ln 2$ con la proposición 5.2, que da $\bar{A} = 0{,}5$ y $\bar{B} = 0{,}5\beta$, tomando $\beta = 2$.

Las dos formas tienen costes complementarios. La recurrencia cuesta $O(n)$ operaciones por token y un estado de tamaño fijo, como una red recurrente, pero es secuencial. La convolución se calcula en paralelo sobre toda la secuencia, por ejemplo con la transformada rápida de Fourier en $O(T \log T)$, que es como S4 entrena [15, §2.4]. Gu y Dao llaman a esta propiedad *invariancia temporal lineal* [14, §2].

5.3 Selección: el modelo de Mamba

La invariancia temporal impide que el modelo decida, según el contenido, qué recordar y qué olvidar: el núcleo $\bar{K}$ trata igual cada token. Gu y Dao argumentan que esta es la razón por la que los SSM anteriores eran débiles en lenguaje, y proponen que los parámetros dependan de la entrada [14, §3.1 y §3.2].

Definición 5.7 (SSM selectivo). Un *SSM selectivo* usa en el instante t las matrices $B_t = s_B(x_t)$, $C_t = s_C(x_t)$ y el paso $\Delta_t = \mathrm{softplus}\big(p + s_\Delta(x_t)\big)$, donde s_B, s_C y s_Δ son proyecciones lineales de la entrada y p es un parámetro, y discretiza con la proposición 5.2 en cada instante [14, §3.2, algoritmo 2]. La matriz A es fija y diagonal.

Teorema 5.8 (Selección y puertas). *Sea un SSM selectivo escalar con $A = -1$, $B = 1$ y $\Delta_t = \mathrm{softplus}(z_t)$, donde z_t es una función lineal de x_t. Su recurrencia es*

$$h_t = (1 - g_t)\, h_{t-1} + g_t\, x_t, \qquad g_t = \sigma(z_t),$$

la de una red recurrente con una puerta que interpola entre conservar el estado y sustituirlo por la entrada [14, §3.5.1, teorema 1].

Demostración. Por la proposición 5.2 con $a = -1$ y $\beta = 1$: $\bar{A}_t = e^{-\Delta_t}$ y $\bar{B}_t = (e^{-\Delta_t} - 1)/(-1) = 1 - e^{-\Delta_t}$. Como $\mathrm{softplus}(z) = \ln(1 + e^z)$,

$$e^{-\Delta_t} = \frac{1}{1 + e^{z_t}} = 1 - \frac{e^{z_t}}{1 + e^{z_t}} = 1 - \sigma(z_t).$$

Luego $\bar{A}_t = 1 - g_t$ y $\bar{B}_t = g_t$, y la recurrencia $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$ es la del enunciado. ∎

Ejemplo 5.9 (Una puerta selectiva). Sean $h_0 = 0$, $(z_1, z_2, z_3) = (0, 2, -2)$ y $x = (1, 3, -1)$. Entonces $g = (0{,}5;\ 0{,}8808;\ 0{,}1192)$ y

$$h_1 = 0{,}5, \quad h_2 = 0{,}1192 \cdot 0{,}5 + 0{,}8808 \cdot 3 = 2{,}702, \quad h_3 = 0{,}8808 \cdot 2{,}702 + 0{,}1192 \cdot (-1) = 2{,}261.$$

Con $z_3 = -2$ el modelo conserva casi todo el estado e ignora casi por completo la entrada x_3; con $z_2 = 2$ hace lo contrario.

5.4 El barrido paralelo

Con parámetros dependientes de t, el coeficiente de x_{t-j} en h_t es $\bar{A}_t \cdots \bar{A}_{t-j+1} \bar{B}_{t-j}$, que depende de t: ya no hay núcleo fijo ni convolución. La linealidad en el estado basta, sin embargo, para paralelizar.

Proposición 5.10 (Asociatividad de la composición de pasos). *Sea $\phi_t(h) = a_t h + b_t$ el paso t de una recurrencia escalar, representado por el par (a_t, b_t). Se define*

$$(a, b) \bullet (a', b') = (a' a,\ a' b + b').$$

Entonces $(\phi_{t'} \circ \phi_t)$ está representado por $(a_t, b_t) \bullet (a_{t'}, b_{t'})$, la operación $\bullet$ es asociativa y, con $h_0 = 0$, h_t es la segunda componente de $(a_1, b_1) \bullet \cdots \bullet (a_t, b_t)$.

Demostración. $\phi_{t'}(\phi_t(h)) = a_{t'}(a_t h + b_t) + b_{t'} = (a_{t'} a_t) h + (a_{t'} b_t + b_{t'})$, que es la afín representada por $(a_t, b_t) \bullet (a_{t'}, b_{t'})$. La asociatividad se sigue de la de la composición de funciones; directamente,

$$\big((a, b) \bullet (a', b')\big) \bullet (a'', b'') = (a'' a' a,\ a'' a' b + a'' b' + b'') = (a, b) \bullet \big((a', b') \bullet (a'', b'')\big).$$

Por último, $h_t = \phi_t \circ \cdots \circ \phi_1(0)$, y la afín $h \mapsto \alpha h + \beta$ evaluada en 0 vale β. ∎

Por la proposición 5.10, los estados $h_1, \dots, h_T$ son los prefijos de un producto asociativo, que un algoritmo de barrido paralelo calcula con profundidad $O(\log T)$ y trabajo $O(T)$. Mamba combina este barrido con la fusión de las operaciones en la memoria rápida de la GPU y el recálculo de los estados intermedios en la pasada hacia atrás, sin escribir nunca el estado expandido en la memoria principal [14, §3.3.2]. Con estado vectorial y A diagonal, cada componente es una recurrencia escalar independiente.

5.5 Comparación con la atención

Cuadro 5.1: Atención causal frente a un SSM selectivo, por capa, con T tokens y dimensión d.

	Atención	SSM selectivo
Coste del entrenamiento	$O(T^2 d)$	$O(Td)$ por cada dimensión de estado
Memoria en la generación	caché que crece con T	estado de tamaño fijo
Coste por token generado	$O(Td)$	independiente de T
Recuperar un dato exacto lejano	directo	limitado por la compresión del estado

Gu y Dao informan de un rendimiento de inferencia unas cinco veces mayor que el de un Transformer de tamaño similar [14, §4.5]. La clase 10 de CS336 señala la debilidad complementaria: los SSM anteriores a Mamba resolvían mal las tareas de recuperación asociativa, importantes en lenguaje, y los diseños recientes son híbridos que conservan algunas capas de atención; Jamba, por ejemplo, intercala capas Transformer y Mamba en proporción $1:7$ dentro de una MoE de $5{,}2 \cdot 10^{10}$ parámetros [16, 20]. No hay información pública de que Claude, Gemini o Grok usen SSM.

Lecturas recomendadas para este tema

- Gu, Goel y Ré [15], §2 (SSM continuo, discretización bilineal y representación convolucional).
- Gu y Dao [14], §2 «State Space Models», §3.2 «Improving SSMs with Selection», §3.3.2 y §3.5.1 «Connection to Gating Mechanisms».
- Stanford CS336 [16], clase 10, sección sobre alternativas al Transformer.
- Seminario CS25 [11], sesión de A. Gu «On the Tradeoffs of State Space Models and Transformers».
- MIT 6.003 [21], sistemas discretos, respuesta al impulso y polos.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ Explique por qué S4 puede entrenarse como una convolución y Mamba no, y qué propiedad de la recurrencia de Mamba permite, aun así, calcularla en paralelo.

Ejercicio 5.2 ⋆⋆ Sea el SSM escalar con $\bar{A} = 0{,}9$, $\bar{B} = C = 1$ y entrada $x = (1, 0, 0, 0, 1)$. Calcule la salida por recurrencia y compruébela con la convolución de la proposición 5.5.

Ejercicio 5.3 ⋆⋆ Sea el sistema escalar con $a = -2$, $\beta = 1$ y $\Delta = 0{,}5$. Calcule $\bar{A}$ y $\bar{B}$ con la retención de orden cero, con la discretización bilineal y con el método de Euler explícito

($\bar{A} = 1 + \Delta a$, $\bar{B} = \Delta\beta$). ¿Cuál de los tres conserva la estabilidad del sistema continuo con cualquier $\Delta > 0$?

Ejercicio 5.4 ⋆⋆ (a) Un sistema discreto cumple $y[n] = 0{,}75\, y[n-1] + x[n-1]$ y está en reposo inicial. Calcule su respuesta al impulso para $n = 0, \dots, 4$ y dé su fórmula cerrada. (b) Se conectan en cascada dos SSM escalares, el primero con $\bar{A} = 1/2$ y el segundo con $\bar{A} = 1/4$, ambos con $\bar{B} = C = 1$. Calcule la respuesta al impulso para $n = 0, \dots, 3$ por recurrencia y compruebe la fórmula $h[n] = 2(1/2)^n - (1/4)^n$.

Fuente: adaptado de MIT 6.003 [21], *Homework* 2, problema 1 «Finding outputs» (sistemas H1 y H3).

Ejercicio 5.5 ⋆⋆ Con el teorema 5.8, $h_0 = 1$, $z = (1, -1, 3)$ y $x = (0, 2, -1)$, calcule Δ_t, g_t y h_t para $t = 1, 2, 3$, y compruebe en $t = 1$ que la discretización de orden cero y la forma con puerta dan el mismo resultado.

Ejercicio 5.6 ⋆⋆⋆ Se busca un SSM cuya respuesta al impulso sea $K = (1, 0, \tfrac{1}{2}, 0, \tfrac{1}{4}, 0, \dots)$, es decir, $K_n = (1/2)^{n/2}$ si n es par y 0 si es impar. (a) Compruebe que la recurrencia $y[n] = \tfrac{1}{2} y[n-2] + x[n]$ la produce. (b) Escríbala como SSM con estado en $\mathbb{R}^2$, $\bar{A} = \begin{pmatrix} 0 & 1/2 \\ 1 & 0 \end{pmatrix}$, $\bar{B} = (1, 0)^\top$ y $C = (1, 0)$; calcule $C\bar{A}^n\bar{B}$ para $n = 0, \dots, 4$ y los autovalores de $\bar{A}$. (c) Demuestre que ningún SSM escalar puede producir esta respuesta.

Fuente: adaptado de MIT 6.003 [21], *Homework* 2, problema 6 «Unit-sample response».

Ejercicio 5.7 ⋆⋆⋆ (a) Con los pares $(a_t, b_t) = (0{,}5;\ 1)$, $(0{,}5;\ 0)$, $(0{,}5;\ 0)$ y $(0{,}5;\ 2)$, calcule h_4 agrupando la composición de la proposición 5.10 en árbol, $\big(e_1 \bullet e_2\big) \bullet \big(e_3 \bullet e_4\big)$, y compare con el ejemplo 5.6. (b) Demuestre que $(1, 0)$ es elemento neutro de $\bullet$ y que $\bullet$ no es conmutativa.

Tema 6

Multimodalidad: imágenes y audio como tokens

Un Transformer opera sobre secuencias de vectores de dimensión d y no depende de cómo se hayan obtenido. Para que procese imágenes o audio basta con una aplicación que convierta la señal en una secuencia de vectores, el análogo de la tabla de *embeddings* para el texto. Este tema estudia las dos construcciones más citadas: el Vision Transformer, que divide la imagen en parches y proyecta cada uno linealmente, y los codificadores de audio, que parten de un espectrograma o de un códec neuronal que produce tokens discretos. Se sigue a Dosovitskiy *et al.* [9, §3.1], la §11.8 de Zhang *et al.* [32], a Radford *et al.* [23, §2.1 y §2.2] y a Zeghidour *et al.* [31], junto con el informe técnico de Gemini 1.5 [12, §3.1].

6.1 El Vision Transformer

Definición 6.1 (Incrustación por parches). Sea una imagen $X \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C_{\text{in}}}$ y sea P un divisor de H_{img} y de W_{img}. Se divide la imagen en $N_p = H_{\text{img}} W_{\text{img}} / P^2$ parches de $P \times P$ píxeles, se aplana cada uno en un vector $x_p^{(k)} \in \mathbb{R}^{1 \times P^2 C_{\text{in}}}$ y se define

$$z_0 = \left[x_{\text{clase}};\ x_p^{(1)} E;\ x_p^{(2)} E;\ \ldots;\ x_p^{(N_p)} E \right] + E_{\text{pos}},$$

con $E \in \mathbb{R}^{P^2 C_{\text{in}} \times d}$, un vector aprendido $x_{\text{clase}} \in \mathbb{R}^{1 \times d}$ y posiciones aprendidas $E_{\text{pos}} \in \mathbb{R}^{(N_p+1) \times d}$ [9, §3.1, ec. 1]. La secuencia z_0, de $N_p + 1$ filas, entra en un Transformer codificador.

Proposición 6.2 (Tokens y coste de un ViT). *Con la notación de la definición 6.1:*

(i) la proyección E tiene $P^2 C_{\text{in}} d$ parámetros, independientemente del tamaño de la imagen;

(ii) si el lado de la imagen se multiplica por κ con P fijo, el número de parches se multiplica por κ^2, el coste de las capas lineales de cada bloque por unos κ^2 y el de los productos de la atención por unos κ^4.

Demostración. (i) E tiene $P^2 C_{\text{in}}$ filas y d columnas. (ii) $N_p' = (\kappa H_{\text{img}})(\kappa W_{\text{img}})/P^2 = \kappa^2 N_p$. Las capas lineales cuestan un número fijo de FLOPs por token, luego escalan con N_p; los productos $QK^\top$ y por V cuestan $O(N_p^2 d)$ y escalan con N_p^2. Las aproximaciones se deben al token de clase, que añade una fila más. ∎

El modelo ViT-Base tiene 12 capas, $d = 768$, un MLP de dimensión 3072 y 12 cabezas, con unos $8{,}6 \cdot 10^7$ parámetros [9, tabla 1]. Con parches de 16 y una imagen de 224×224, la secuencia tiene $196 + 1 = 197$ tokens y E tiene $768 \cdot 768 = 589\,824$ parámetros.

Cuadro 6.1: Número de parches y dimensión de cada parche aplanado para imágenes RGB ($C_{\text{in}} = 3$).

Imagen	Parche	N_p	$P^2 C_{\text{in}}$
224×224	16×16	$14^2 = 196$	768
384×384	16×16	$24^2 = 576$	768
224×224	32×32	$7^2 = 49$	3072

Observación 6.3 (Parches y convoluciones). Aplicar E a cada parche es lo mismo que aplicar una convolución con d filtros de tamaño $P \times P$ y paso P: cada filtro es una columna de E reordenada como un tensor $P \times P \times C_{\text{in}}$, y el paso igual al tamaño hace que los parches no se solapen. Es la forma en que suele implementarse.

6.2 Audio

El audio es una señal unidimensional muestreada a miles de muestras por segundo: a 16 kHz, treinta segundos son 480 000 muestras, demasiadas para usarlas como tokens. Hay dos formas habituales de reducirlas.

Ejemplo 6.4 (El codificador de Whisper). Whisper remuestrea el audio a 16 kHz y lo divide en fragmentos de 30 s [23, §2.1]. Sobre cada fragmento calcula un espectrograma log-Mel de 80 canales con ventanas de 25 ms y desplazamiento de 10 ms, lo que da $30/0{,}01 = 3000$ vectores de dimensión 80. Dos convoluciones unidimensionales de anchura 3 con activación GELU, la segunda con paso 2, reducen la secuencia a 1500 vectores, a los que se suman posiciones sinusoidales antes de entrar en el codificador Transformer [23, §2.2].

La segunda vía convierte el audio en enteros. Un *códec neuronal* como SoundStream [31] o EnCodec [8] consta de un codificador convolucional que produce un vector por cada fotograma de unos milisegundos, un cuantizador que sustituye ese vector por índices de diccionarios finitos y un decodificador que reconstruye la onda. SoundStream usa un cuantizador vectorial residual: el primer diccionario aproxima el vector, el segundo cuantiza el error del primero, y así sucesivamente.

Proposición 6.5 (Tasa de un cuantizador vectorial residual). *Un códec que produce ν fotogramas por segundo y cuantiza cada uno con N_q diccionarios de K entradas genera νN_q tokens por segundo y una tasa de $\nu N_q \log_2 K$ bits por segundo.*

Demostración. Cada fotograma se representa por N_q índices, uno por diccionario, y cada índice elige entre K valores, lo que requiere $\log_2 K$ bits. ■

Con valores ilustrativos, $\nu = 50$ Hz, $N_q = 8$ y $K = 1024$, se obtienen 400 tokens por segundo y 4 kbit/s, dentro del intervalo de 3 a 18 kbit/s que cubre SoundStream [31]. Con estos tokens, el audio se modela con el mismo objetivo de predicción del siguiente token del módulo 1.

6.3 Fusión con el modelo de lenguaje

Hay dos formas de unir los vectores de otra modalidad con los de texto. En la *fusión con adaptador*, un codificador separado (un ViT, por ejemplo) produce vectores que una proyección lineal o un pequeño MLP lleva a la dimensión d del modelo de lenguaje, y se insertan en la secuencia de texto. En la *fusión nativa*, el modelo se entrena desde el principio con secuencias en las que se intercalan tokens de varias modalidades.

Observación 6.6 (Lo que es público). El informe de Gemini 1.5 afirma que el modelo es multimodal de forma nativa y admite datos de distintas modalidades intercalados en una misma secuencia: texto, imágenes, audio y vídeo, con ejemplos de 107 horas de audio y 10,5 horas de vídeo a un fotograma por segundo [12, §3.1]. No describe sus codificadores. Claude y las versiones de Grok posteriores a Grok-1 aceptan imágenes según la documentación pública de sus productos, pero sus codificadores tampoco son públicos; Grok-1, el único con pesos publicados, es un modelo de texto [30]. Lo que sí puede afirmarse en general es que, sea cual sea el codificador, las demás modalidades acaban como vectores en el mismo flujo residual que el texto.

Lecturas recomendadas para este tema

- Dosovitskiy *et al.* [9], §3.1 «Vision Transformer (ViT)», ecuaciones 1 a 4 y tabla 1.
- Zhang *et al.* [32], §11.8 «Transformers for Vision».
- Radford *et al.* [23], §2.1 «Data Processing» y §2.2 «Model».
- Zeghidour *et al.* [31] y Défossez *et al.* [8]: códecs neuronales de audio.
- Seminario CS25 [11], sesión de V. Lin sobre modelos multimodales nativos; temario de CMU 11-711 [4], bloque de multimodalidad.

Ejercicios del tema 6

Ejercicio 6.1 ⋆ ¿Qué se gana y qué se pierde al pasar de parches de 16×16 a parches de 32×32 en una imagen de 224×224? Cuantifique la variación del número de tokens y del coste de la atención.

Ejercicio 6.2 ⋆⋆ ¿Cuántos parches produce un ViT con parches de 16 para imágenes de 224×224 y de 384×384? ¿Por qué factor crecen el coste de las capas lineales y el de la atención?

Ejercicio 6.3 ⋆⋆ Un ViT con parches de 16×16, token de clase, $d = 512$ y MLP de dimensión 2048 recibe imágenes de 96×96 o de 192×192. (a) Cuente los tokens n en cada caso. (b) Estime los FLOPs por bloque de las capas lineales, $n(8d^2 + 4d\,d_{\text{ff}})$, y de la atención, $4n^2d$. (c) ¿Cuánto crece el coste por bloque al duplicar el lado y qué fracción corresponde a la atención en cada caso? (d) ¿A partir de qué n la atención costaría más que las capas lineales?

Fuente: adaptado de Zhang *et al.* [32], §11.8 «Transformers for Vision», ejercicio 1 (efecto del tamaño de la imagen en el coste).

Ejercicio 6.4 ⋆⋆ Con los datos del ejemplo 6.4: (a) ¿cuántas muestras tiene un fragmento? (b) ¿Cuántos vectores salen del espectrograma y cuántos de las convoluciones? (c) ¿Cuántos

vectores procesa el codificador para una grabación de 10 minutos? (d) Compare con un códec de $\nu = 50$ Hz y $N_q = 8$ diccionarios que se modelase token a token.

Ejercicio 6.5 ⋆⋆ Para ViT-Base con parches de 16 sobre imágenes RGB de 224×224, calcule los parámetros de E, de E_{pos} y del token de clase, y la fracción que suponen sobre los $8{,}6 \cdot 10^7$ parámetros del modelo.

Ejercicio 6.6 ⋆⋆⋆ (a) Demuestre que la incrustación por parches de la definición 6.1 coincide con una convolución bidimensional con d filtros de tamaño $P \times P \times C_{\text{in}}$ y paso P. (b) Demuestre que una convolución de tamaño P y paso $S \le P$ sobre una imagen de lado ℓ, con S divisor de $\ell - P$, produce $\big((\ell - P)/S + 1\big)^2$ vectores, y evalúe el resultado con $\ell = 224$, $P = 16$ y $S = 8$.

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; los recuentos de parámetros y de bytes son enteros exactos. Se usan las unidades binarias de la presentación: 1 KiB = 1024 bytes, 1 MiB = 1024^2 bytes y 1 GiB = $1024^3 = 1\,073\,741\,824$ bytes.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. tres afirmaciones sobre RMSNorm, SwiGLU y la regla de las cabezas.

Planteamiento. contrastar cada una con la proposición 1.4, la proposición 1.8 y la cuadro 1.2.

Desarrollo.

(a) **Falsa.** RMSNorm no resta la media. Con $x = (1, 0)$ y $t = 1$: $\mathrm{RMS}(x) = \sqrt{1/2}$ y $\mathrm{RMSNorm}(x) = (1{,}414;\ 0)$, mientras que $x + t\mathbf{1} = (2, 1)$ tiene RMS $= \sqrt{5/2} = 1{,}581$ y $\mathrm{RMSNorm}(x + t\mathbf{1}) = (1{,}265;\ 0{,}6325)$. La invariancia ante traslaciones es propia de LayerNorm.

(b) **Verdadera.** SwiGLU con $d_{\mathrm{ff}} = 4d$ tiene $3{\cdot}d{\cdot}4d = 12d^2$ parámetros, frente a $2{\cdot}d{\cdot}4d = 8d^2$: $12/8 = 1{,}5$. Por eso se reduce d_{ff} a $\frac{8}{3}d$.

(c) **Verdadera.** Llama 3 8B: $32 \cdot 128 = 4096$; 70B: $64 \cdot 128 = 8192$; 405B: $128 \cdot 128 = 16\,384$; Mixtral: $32 \cdot 128 = 4096$; Grok-1: $48 \cdot 128 = 6144$.

Resultado. **(a) falsa; (b) verdadera; (c) verdadera.**

Comprobación. en (a), $\|(1{,}265;\ 0{,}6325)\|^2 = 1{,}600 + 0{,}4000 = 2 = d$, como exige la proposición 1.4(i).

Solución del ejercicio 1.2

Datos. $x = (2, -1, 0, 3)$, $d = 4$, $\gamma = \mathbf{1}$, $\beta = 0$, $\varepsilon = 0$.

Planteamiento. aplicar la definición 1.3; para $3x$, la proposición 1.4(ii).

Desarrollo.

1. $\sum_i x_i^2 = 4 + 1 + 0 + 9 = 14$; $\mathrm{RMS}(x) = \sqrt{14/4} = \sqrt{3{,}5} = 1{,}871$.
2. $\mathrm{RMSNorm}(x) = (2, -1, 0, 3)/1{,}871 = (1{,}069;\ -0{,}5345;\ 0;\ 1{,}604)$.
3. LayerNorm: $\mu(x) = (2{-}1{+}0{+}3)/4 = 1$; $c = x - \mathbf{1} = (1, -2, -1, 2)$; $\sum_i c_i^2 = 1{+}4{+}1{+}4 = 10$; $\mathrm{RMS}(c) = \sqrt{2{,}5} = 1{,}581$; $\mathrm{LN}(x) = (0{,}6325;\ -1{,}265;\ -0{,}6325;\ 1{,}265)$.

4. RMS$(3x) = 3 \cdot 1{,}871 = 5{,}612$, luego RMSNorm$(3x) = (6, -3, 0, 9)/5{,}612$, que vale $(1{,}069;\ -0{,}5345;\ 0;\ 1{,}604) =$ RMSNorm(x).
5. Norma: $1{,}069^2 + 0{,}5345^2 + 0 + 1{,}604^2 = 1{,}143 + 0{,}2857 + 2{,}571 = 4{,}000 = d$.

Resultado. RMSNorm(x) = RMSNorm$(3x) = (1{,}069;\ -0{,}5345;\ 0;\ 1{,}604)$ **y** LN$(x) = (0{,}6325;\ -1{,}265;\ -0{,}6325;\ 1{,}265)$**.**

Comprobación. la salida de LayerNorm tiene media 0 y norma al cuadrado $0{,}4+1{,}6+0{,}4+1{,}6 = 4 = d$, como la de RMSNorm aplicada al vector centrado (proposición 1.4(iv)).

Solución del ejercicio 1.3

Datos. cuadro 1.2: $(L, d, d_{\text{ff}}, H, n_{\text{kv}}) = (32, 4096, 14\,336, 32, 8)$, $(80, 8192, 28\,672, 64, 8)$ y $(126, 16\,384, 53\,248, 128, 8)$.

Planteamiento. $d_h = d/H$; cabezas por grupo H/n_{kv}; bloque $= 2d^2$ $(W_Q,\ W_O)$ $+ 2d\, n_{\text{kv}} d_h$ $(W_K,\ W_V)$ $+ 3d\, d_{\text{ff}}$ (SwiGLU) $+ 2d$ (normas).

Desarrollo.

1. Dimensión por cabeza: $4096/32 = 8192/64 = 16\,384/128 = 128$ en los tres.
2. Consultas por cabeza K/V: $32/8 = 4$, $64/8 = 8$ y $128/8 = 16$.
3. 8B: atención $2 \cdot 4096^2 + 2 \cdot 4096 \cdot 1024 = 33\,554\,432 + 8\,388\,608 = 41\,943\,040$; MLP $3 \cdot 4096 \cdot 14\,336 = 176\,160\,768$; normas 8192; bloque 218 112 000.
4. 70B: atención $2 \cdot 8192^2 + 2 \cdot 8192 \cdot 1024 = 150\,994\,944$; MLP $3 \cdot 8192 \cdot 28\,672 = 704\,643\,072$; normas 16 384; bloque 855 654 400.
5. 405B: atención $2 \cdot 16\,384^2 + 2 \cdot 16\,384 \cdot 1024 = 570\,425\,344$; MLP $3 \cdot 16\,384 \cdot 53\,248 = 2\,617\,245\,696$; normas 32 768; bloque 3 187 703 808.
6. Fracción del MLP en 8B: $176\,160\,768/218\,112\,000 = 0{,}8077$.

Resultado. $d_h = 128$ **en los tres; 4, 8 y 16 consultas por cabeza K/V; bloques de** 218 112 000**,** 855 654 400 **y** 3 187 703 808 **parámetros; el MLP es el** 80,77 % **del bloque de 8B.**

Comprobación. los bloques del modelo 8B suman $32 \cdot 218\,112\,000 = 6{,}980 \cdot 10^9$; con *embeddings* de entrada y salida de $128\,256 \times 4096$ (unos $1{,}05 \cdot 10^9$) el total queda cerca de los $8 \cdot 10^9$ del nombre.

Solución del ejercicio 1.4

Datos. $d = 512$; SwiGLU con $d_{\text{ff}} \approx \frac{8}{3}d$ redondeado a múltiplo de 64; SiLU con $d_{\text{ff}} = 4d$.

Planteamiento. SwiGLU tiene $3d\, d_{\text{ff}}$ parámetros y el MLP SiLU, $2d \cdot 4d$ (proposición 1.8).

Desarrollo.

1. $\frac{8}{3} \cdot 512 = 1365{,}3$; $1365{,}3/64 = 21{,}33$, cuyo entero más próximo es 21; $d_{\text{ff}} = 21 \cdot 64 = 1344$.
2. SwiGLU: $3 \cdot 512 \cdot 1344 = 2\,064\,384$.
3. SiLU: $2 \cdot 512 \cdot 2048 = 2\,097\,152$.
4. Diferencia relativa: $(2\,097\,152 - 2\,064\,384)/2\,097\,152 = 32\,768/2\,097\,152 = 0{,}015\,625 = 1{,}563\,\%$.

Resultado. $d_{\text{ff}} = 1344$**; 2 064 384 frente a 2 097 152 parámetros; el SwiGLU tiene un** 1,563 % **menos.** La comparación es justa porque ambos MLP tienen casi los mismos parámetros y FLOPs; una ventaja del SwiGLU no puede atribuirse a un mayor tamaño.

Comprobación. sin redondeo, $3 \cdot 512 \cdot 1365{,}3 = 2\,097\,152$, exactamente igual al MLP SiLU; toda la diferencia procede del redondeo $1365{,}3 \to 1344$.

Solución del ejercicio 1.5

Datos. $x \in \mathbb{R}^d$ no constante, $t \in \mathbb{R}$, $\varepsilon = 0$, $\beta = 0$.

Planteamiento. usar $\mu(x + t\mathbf{1}) = \mu(x) + t$ y la proposición 1.4(iv).

Desarrollo.

1. $\mu(x + t\mathbf{1}) = \frac{1}{d}\sum_i (x_i + t) = \mu(x) + t$, luego $(x + t\mathbf{1}) - \mu(x + t\mathbf{1})\mathbf{1} = x - \mu(x)\mathbf{1}$.
2. Por la proposición 1.4(iv), $\mathrm{LN}(x+t\mathbf{1}) = \mathrm{RMSNorm}(x-\mu(x)\mathbf{1}) = \mathrm{LN}(x)$; $x+t\mathbf{1}$ tampoco es constante.
3. Contraejemplo para RMSNorm: $x = (1, -1)$, $t = 1$. $\mathrm{RMSNorm}(x) = (1, -1)$, porque $\mathrm{RMS}(x) = 1$. $x + t\mathbf{1} = (2, 0)$, $\mathrm{RMS} = \sqrt{2}$ y $\mathrm{RMSNorm}(x + t\mathbf{1}) = (1{,}414;\ 0) \neq (1, -1)$.
4. Recuperación salvo escala: si $\gamma = \mathbf{1}$, $u = \mathrm{RMSNorm}(x) = x/\,\mathrm{RMS}(x)$, luego $x = \mathrm{RMS}(x)\,u$ con $\mathrm{RMS}(x) > 0$: x es un múltiplo positivo de u. Recíprocamente, todo múltiplo positivo de u da la misma salida por la proposición 1.4(ii).

Resultado. **LayerNorm es invariante ante traslaciones y RMSNorm no; la salida de RMSNorm determina la entrada exactamente salvo un factor positivo.** ■

Comprobación. LayerNorm pierde más información: determina x salvo un factor positivo y una traslación, es decir, pierde dos grados de libertad frente a uno de RMSNorm.

Solución del ejercicio 1.6

Datos. (a) $d_{\text{ff}} = \rho d$, n_{kv} cabezas K/V, $d_h = d/H$; (b) $N_{\text{b}} = 7 \cdot 10^{10}$, $r = 102$; (c) Llama 3 70B: $L = 80$, $d = 8192$, $H = 64$, $n_{\text{kv}} = 8$, $\rho = 3{,}5$.

Planteamiento. repetir el recuento de la proposición 1.10 con las nuevas matrices.

Desarrollo.

1. (a) Atención: W_Q, W_O suman $2d^2$; W_K, W_V tienen $d \times n_{\text{kv}} d_h = d \times (n_{\text{kv}}/H)d$ cada una, en total $2(n_{\text{kv}}/H)d^2$. MLP: $3\rho d^2$. Bloque: $c\,d^2$ con $c = 2 + 2n_{\text{kv}}/H + 3\rho$. Con $L = d/r$: $N_{\text{b}} = c\,d^3/r$ y $d = (rN_{\text{b}}/c)^{1/3}$. Con $n_{\text{kv}} = H$ y $\rho = 8/3$, $c = 2 + 2 + 8 = 12$.
2. (b) $d = (102 \cdot 7 \cdot 10^{10}/12)^{1/3} = (5{,}950 \cdot 10^{11})^{1/3} = 8411$; $L = 8411/102 = 82{,}46$. Llama 3 70B tiene $d = 8192$ y $L = 80$.
3. (c) $c = 2+2\cdot 8/64+3\cdot 3{,}5 = 2+0{,}25+10{,}5 = 12{,}75$; $N_{\text{b}} = 80\cdot 8192^2 \cdot 12{,}75 = 68\,451\,041\,280$.
4. Con $c = 12{,}75$, la fórmula de (a) da $d = (102 \cdot 7 \cdot 10^{10}/12{,}75)^{1/3} = 8243$, más cerca de 8192.

Resultado. **(a)** $d = (rN_{\text{b}}/c)^{1/3}$ **con** $c = 2+2n_{\text{kv}}/H+3\rho$**; (b)** $d \approx 8411$**,** $L \approx 82$**; (c)** $c = 12{,}75$ **y** $N_{\text{b}} = 68\,451\,041\,280$**.** La diferencia de (b) se debe a que Llama 3 ahorra en W_K, W_V ($0{,}25d^2$ en lugar de $2d^2$) y gasta más en el MLP ($10{,}5d^2$ en lugar de $8d^2$).

Comprobación. el recuento exacto del ejercicio 1.3, $80 \cdot 855\,654\,400 = 68\,452\,352\,000$, difiere de (c) en $1\,310\,720 = 80 \cdot 2 \cdot 8192$, que son las normalizaciones despreciadas.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. tres afirmaciones sobre GQA, MQA y MLA.

Planteamiento. contrastar con la proposición 2.6, la proposición 2.2 y la definición 2.8.

Desarrollo.

(a) **Falsa.** GQA conserva las H cabezas de consulta; W_Q sigue siendo $d \times d$ y sus FLOPs no cambian (proposición 2.6(ii)).

(b) **Verdadera.** Con $n_{\text{kv}} = 1$, la caché por token y capa es $2d_h s$ bytes, sin H en la fórmula; en bf16 con $d_h = 128$, 512 bytes.

(c) **Falsa.** MLA guarda un único vector latente c_t de dimensión d_c por token y capa (más la clave de RoPE desacoplada, compartida), no claves y valores por cabeza.

Resultado. **(a) falsa; (b) verdadera; (c) falsa.**

Comprobación. en (c), con los datos de DeepSeek-V2, MLA guarda 576 números por token y capa, frente a $2 \cdot 128 \cdot 128 = 32\,768$ si guardase claves y valores de todas las cabezas.

Solución del ejercicio 2.2

Datos. $L = 32$, $H = 32$, $d_h = 128$, $s = 2$, $T = 8192$, $b = 1$; $n_{\text{kv}} = 32$, 8 y 1.

Planteamiento. proposición 2.2: $M_{\text{KV}} = 2Ln_{\text{kv}}d_hTbs$.

Desarrollo.

1. MHA por token: $2{\cdot}32{\cdot}32{\cdot}128{\cdot}2 = 524\,288$ bytes; total $524\,288{\cdot}8192 = 4\,294\,967\,296$ bytes $= 4$ GiB.
2. GQA-8 por token: $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131\,072$ bytes; total $1\,073\,741\,824$ bytes $= 1$ GiB.
3. MQA por token: $2 \cdot 32 \cdot 1 \cdot 128 \cdot 2 = 16\,384$ bytes; total $134\,217\,728$ bytes $= 128$ MiB.

Resultado. **MHA 4 GiB; GQA-8 1 GiB; MQA 128 MiB.**

Comprobación. los cocientes son $4/1 = 4 = 32/8$ y $4096/128 = 32 = 32/1$, es decir, H/G (proposición 2.6(i)).

Solución del ejercicio 2.3

Datos. $V = 32\,000$, $d = 5120$, $d_{\text{ff}} = 13\,824$, $L = 40$, $H = 40$, $d_h = 128$, $T = 1024$, $b = 64$, $s = 2$, ancho de banda $3{,}35 \cdot 10^{12}$ bytes/s.

Planteamiento. parámetros $= 2Vd + 3dd_{\text{ff}}L + (2dHd_h + 2dn_{\text{kv}}d_h)L$; memoria $= 2{\cdot}$parámetros $+\, b\,M_{\text{KV}}$; latencia $=$ memoria/ancho de banda; rendimiento $= b/$latencia.

Desarrollo.

1. Partes comunes: $2Vd = 327\,680\,000$; $3dd_{\text{ff}}L = 8\,493\,465\,600$; $2dHd_hL = 2\,097\,152\,000$.
2. (a) $n_{\text{kv}} = 40$: $2dn_{\text{kv}}d_hL = 2\,097\,152\,000$; parámetros $13\,015\,449\,600$; pesos en bf16, $26\,030\,899\,200$ bytes.
3. Caché por secuencia: $2 \cdot 40 \cdot 40 \cdot 128 \cdot 1024 \cdot 2 = 838\,860\,800$ bytes; con $b = 64$, $53\,687\,091\,200$ bytes.

4. Memoria total: 79 717 990 400 bytes ≈ 79,72 GB; latencia $79{,}72 \cdot 10^9 / 3{,}35 \cdot 10^{12} = 0{,}023\,80$ s = 23,80 ms; rendimiento $64/0{,}023\,80 = 2689$ tokens/s.
5. (b) $n_{\text{kv}} = 8$: $2dn_{\text{kv}}d_hL = 419\,430\,400$; parámetros 11 337 728 000; pesos 22 675 456 000 bytes.
6. Caché por secuencia: 167 772 160 bytes; con $b = 64$, 10 737 418 240 bytes.
7. Memoria total 33 412 874 240 bytes ≈ 33,41 GB; latencia 9,974 ms; rendimiento 6417 tokens/s.

Resultado. **(a)** $1{,}302 \cdot 10^{10}$ **parámetros,** 79,72 **GB,** 23,80 **ms por paso y** 2689 **tokens/s; (b)** $1{,}134 \cdot 10^{10}$ **parámetros,** 33,41 **GB,** 9,974 **ms y** 6417 **tokens/s.** GQA reduce la caché a la quinta parte y, en este régimen limitado por la memoria, multiplica el rendimiento por 2,386.

Comprobación. la caché se divide exactamente por $40/8 = 5$: $53\,687\,091\,200/10\,737\,418\,240 = 5$. La clase de CS336 añade que el ahorro permite subir el lote a 256 (65,63 GB y 13 068 tokens/s), mientras que con MHA no cabría en 80 GB.

Solución del ejercicio 2.4

Datos. Llama 2 70B: $H = 64$; $n_{\text{kv}} = 64$ (MHA), 8 (GQA) o 1 (MQA); $s = 2$; $I^* = 156$ FLOP/byte.

Planteamiento. proposición 2.4: $I = 2H/(sn_{\text{kv}})$; según el modelo *roofline*, la fracción del máximo es $\min(1, I/I^*)$.

Desarrollo.

1. MHA: $I = 2 \cdot 64/(2 \cdot 64) = 1$; fracción $1/156 = 0{,}6410\,\%$.
2. GQA-8: $I = 128/16 = 8$; fracción $8/156 = 5{,}128\,\%$.
3. MQA: $I = 128/2 = 64$; fracción $64/156 = 41{,}03\,\%$.

Resultado. $I = 1$**,** 8 **y** 64 **FLOP/byte; como máximo un** 0,6410 %**, un** 5,128 % **y un** 41,03 % **del rendimiento de la A100.** Las tres variantes siguen limitadas por la memoria.

Comprobación. los cocientes entre intensidades son $8 = H/G$ y $64 = H/1$, como afirma la proposición 2.6(iii).

Solución del ejercicio 2.5

Datos. $L = 40$, $H = 32$, $d_h = 128$, $d_c = 256$, $d_h^R = 64$, $s = 2$, presupuesto 16 GiB.

Planteamiento. cuadro 2.1: MLA $(d_c + d_h^R)L$ y MHA $2Hd_hL$ números por token; grupos equivalentes $G = (d_c + d_h^R)/(2d_h)$.

Desarrollo.

1. MLA: $(256 + 64) \cdot 40 = 12\,800$ números; 25 600 bytes = 25 KiB por token.
2. MHA: $2 \cdot 32 \cdot 128 \cdot 40 = 327\,680$ números; 655 360 bytes = 640 KiB por token. Cociente $327\,680/12\,800 = 25{,}6$.
3. Grupos equivalentes: $320/256 = 1{,}25$.
4. Tokens en 16 GiB: MLA, $17\,179\,869\,184/25\,600 = 671\,088{,}64$, es decir, 671 088; MHA, $17\,179\,869\,184/655\,360 = 26\,214{,}4$, es decir, 26 214.

Resultado. **(a) 25 KiB frente a 640 KiB por token (25,6 veces menos); (b) 1,25 grupos; (c) unos 671 088 tokens con MLA y 26 214 con MHA.**

Comprobación. 671 088,64/26 214,4 = 25,6, el mismo cociente que en el paso 2.

Solución del ejercicio 2.6

Datos. grupo $\mathcal{G}_g$ de $m = H/G$ cabezas con $W_K^{(i)} = W_K^*$ y $W_V^{(i)} = W_V^*$ para $i \in \mathcal{G}_g$.

Planteamiento. la conversión de Ainslie *et al.* asigna al grupo la media de sus proyecciones [1, §2.1].

Desarrollo.

1. (a) $W_K^{(g)} = \frac{1}{m}\sum_{i\in\mathcal{G}_g} W_K^{(i)} = \frac{1}{m}\, m\, W_K^* = W_K^*$, y lo mismo para W_V.
2. Cada cabeza i del grupo calcula las mismas claves XW_K^* y valores XW_V^* antes y después de la conversión, y su consulta $XW_Q^{(i)}$ no cambia; las puntuaciones, los pesos y la salida de la cabeza son idénticos.
3. (b) Sean $d = d_h = 1$, dos cabezas con $W_Q^{(1)} = W_Q^{(2)} = 1$, $W_K^{(1)} = 1$ y $W_K^{(2)} = 3$, una consulta procedente de una entrada igual a 1 (luego $q = 1$ en ambas cabezas) y dos posiciones previas con entradas $x_1 = 1$ y $x_2 = 2$. Antes de convertir, la cabeza 1 tiene claves 1 y 2 y puntuaciones 1 y 2; la cabeza 2, claves 3 y 6 y puntuaciones 3 y 6. Tras promediar, $W_K = (1{+}3)/2 = 2$, las claves son 2 y 4 en ambas cabezas y las puntuaciones, 2 y 4.

Resultado. **(a) si las proyecciones de cada grupo coinciden, la conversión es exacta; (b) en general cambia las puntuaciones, por lo que hace falta el ajuste posterior (*uptraining*).** ■

Comprobación. en (b), las diferencias de puntuación entre las dos posiciones pasan de 1 y 3 a 2 y 2: los pesos de la softmax cambian en ambas cabezas.

Solución del ejercicio 2.7

Datos. vectores fila; $k_{j,i} = c_j W_i^{UK} R_j$, consulta $q_{t,i} R_t$; $R_m R_n^\top = R_{m-n}$; frecuencias $0 < \theta_l < 2\pi$.

Planteamiento. desarrollar la traspuesta del producto y analizar cuándo $R_\delta (W_i^{UK})^\top$ no depende de δ.

Desarrollo.

1. (a) $q_{t,i} R_t (c_j W_i^{UK} R_j)^\top = q_{t,i} R_t R_j^\top (W_i^{UK})^\top c_j^\top = q_{t,i} R_{t-j} (W_i^{UK})^\top c_j^\top$. Luego $M_{t,j} = R_{t-j}(W_i^{UK})^\top \in \mathbb{R}^{d_h \times d_c}$.
2. (b) Si $M_{t,j}$ es la misma para todos los pares, en particular $R_1 (W_i^{UK})^\top = R_0 (W_i^{UK})^\top = (W_i^{UK})^\top$. Cada columna w de $(W_i^{UK})^\top$ cumple $R_1 w = w$.
3. R_1 es diagonal por bloques con rotaciones de ángulo θ_l. Una rotación plana de ángulo $\theta_l \notin 2\pi\mathbb{Z}$ solo fija el vector nulo, porque $R(\theta)w = w$ con $w \neq 0$ exige que 1 sea autovalor de $R(\theta)$, cuyos autovalores son $e^{\pm i\theta} \neq 1$.
4. Por tanto, cada par de componentes de w es nulo, $w = 0$, y $W_i^{UK} = 0$.
5. Con $W_i^{UK} \neq 0$, la matriz que relaciona $q_{t,i}$ con c_j cambia con la distancia $t - j$; no puede precalcularse una única $W_i^Q (W_i^{UK})^\top$ que sirva para todas las posiciones de la caché.

Resultado. $M_{t,j} = R_{t-j}(W_i^{UK})^\top$**, que solo es constante si** $W_i^{UK} = 0$**; por eso MLA añade una clave rotada aparte.** ■

Comprobación. sin RoPE, $R_m = I$ para todo m y $M_{t,j} = (W_i^{UK})^\top$, que es la matriz absorbida en la proposición 2.9.

A.3 Tema 3

Solución del ejercicio 3.1

Datos. tres afirmaciones sobre la ventana deslizante y la interpolación de posiciones.

Planteamiento. contrastar con la definición 3.1, la proposición 3.7 y la definición 3.10.

Desarrollo.

(a) **Verdadera.** La caché circular tiene W ranuras por capa (proposición 3.7); su tamaño, $2n_{\text{kv}}d_h W s$ bytes, no contiene T.

(b) **Verdadera.** PI divide por s_{e} el ángulo $m\theta_i$ de todos los pares por igual; el cambio de base NTK divide θ_i por $s_{\text{e}}^{2i/(d_h-2)}$, un factor que va de 1 en el par más rápido a s_{e} en el más lento (proposición 3.12).

(c) **Falsa.** La posición i atiende a $j \in [i - W + 1, i]$; la posición $i - W$ queda fuera. Solo puede influir a través de una segunda capa.

Resultado. **(a) verdadera; (b) verdadera; (c) falsa.**

Comprobación. en (c), el teorema 3.3 con $L = 1$ da un campo receptivo de W posiciones, de $i - W + 1$ a i.

Solución del ejercicio 3.2

Datos. $T = 16\,384$, $W = 4096$.

Planteamiento. proposición 3.2.

Desarrollo.

1. Completa: $T(T+1)/2 = 16\,384 \cdot 16\,385/2 = 268\,451\,840/2 = 134\,225\,920$.
2. Ventana, filas 1 a W: $W(W+1)/2 = 4096 \cdot 4097/2 = 8\,390\,656$.
3. Filas $W+1$ a T: $(T-W)W = 12\,288 \cdot 4096 = 50\,331\,648$.
4. Total con ventana: $8\,390\,656 + 50\,331\,648 = 58\,722\,304$.
5. Cociente: $134\,225\,920/58\,722\,304 = 2{,}286$; la aproximación da $T/(2W) = 2$.

Resultado. $134\,225\,920$ **frente a** $58\,722\,304$ **pares; cociente** $2{,}286$**, frente a** 2 **de la aproximación.**

Comprobación. con la fórmula cerrada, $TW - W(W-1)/2 = 67\,108\,864 - 8\,386\,560 = 58\,722\,304$. La aproximación $T/(2W)$ subestima porque $T/W = 4$ no es grande; con $T = 32\,768$ el cociente exacto es 4,267 y la aproximación, 4.

Solución del ejercicio 3.3

Datos. $W = 4$; posiciones $0, \ldots, 9$.

Planteamiento. definición 3.6: la posición i va a la ranura $i \bmod 4$.

Desarrollo.

1. Ranuras: $0 \to 0$, $1 \to 1$, $2 \to 2$, $3 \to 3$, $4 \to 0$, $5 \to 1$, $6 \to 2$, $7 \to 3$, $8 \to 0$, $9 \to 1$.
2. Contenido final: la ranura 0 guarda la última posición escrita en ella, la 8; la ranura 1, la 9; la ranura 2, la 6; la ranura 3, la 7.
3. La posición 9 necesita $j \in [9-4+1, 9] = \{6, 7, 8, 9\}$, que es exactamente el contenido de la caché.

Resultado. **tras la posición 9, las ranuras** $(0, 1, 2, 3)$ **contienen las posiciones** $(8, 9, 6, 7)$**, que son las de su ventana.**

Comprobación. las posiciones $6, 7, 8, 9$ tienen restos $2, 3, 0, 1$ módulo 4, todos distintos, como exige la demostración de la proposición 3.7.

Solución del ejercicio 3.4

Datos. $d_h = 128$, $\beta_0 = 10^4$, $s_\mathrm{e} = 4$.

Planteamiento. definición 3.11 y proposición 3.12: $\beta_0' = \beta_0 s_\mathrm{e}^{128/126}$ y $\theta_i' = \theta_i s_\mathrm{e}^{-2i/126}$; PI divide todas por s_e.

Desarrollo.

1. $s_\mathrm{e}^{128/126} = 4^{1{,}016} = 4{,}089$; $\beta_0' = 40\,890$.
2. Originales: $\theta_0 = 1$; $\theta_{32} = 10^{4\cdot(-64/128)} = 10^{-2} = 0{,}01$; $\theta_{63} = 10^{-4\cdot 126/128} = 1{,}155 \cdot 10^{-4}$.
3. PI: 0,25; 0,0025; $2{,}887 \cdot 10^{-5}$.
4. NTK: $\theta_0' = 1$; $\theta_{32}' = 0{,}01 \cdot 4^{-64/126} = 0{,}01/2{,}022 = 0{,}004\,945$; $\theta_{63}' = 1{,}155 \cdot 10^{-4}/4 = 2{,}887 \cdot 10^{-5}$.
5. (c) Para θ_{32}, PI divide por 4 y NTK por $2{,}022 \approx \sqrt{4}$.
6. (d) $\sqrt{1/t} = 0{,}1 \ln 4 + 1 = 0{,}1 \cdot 1{,}386 + 1 = 1{,}139$.

Resultado. $\beta_0' = 40\,890$**; con NTK,** θ_0 **no cambia,** θ_{63} **se divide por 4 como en PI y** θ_{32} **se divide por** 2,022 **en lugar de por 4;** $\sqrt{1/t} = 1{,}139$**.**

Comprobación. el par central ($i = 32$, a mitad del recorrido $0 \ldots 63$) se divide por $s_\mathrm{e}^{64/126} \approx s_\mathrm{e}^{1/2}$: el cambio de base reparte la interpolación geométricamente entre 1 y s_e.

Solución del ejercicio 3.5

Datos. $n = 4096$, $d = 1024$, $k = 3$.

Planteamiento. sustituir en $n^2 d$, $n d^2$ y $k n d^2$; el cociente atención/recurrente es n/d.

Desarrollo.

1. Atención: $4096^2 \cdot 1024 = 17\,179\,869\,184 \approx 1{,}718 \cdot 10^{10}$.
2. Recurrente: $4096 \cdot 1024^2 = 4\,294\,967\,296 \approx 4{,}295 \cdot 10^{9}$.
3. Convolución: $3 \cdot 4\,294\,967\,296 = 12\,884\,901\,888 \approx 1{,}288 \cdot 10^{10}$.
4. $n^2 d > n d^2 \iff n > d = 1024$.

5. La convolución conecta posiciones a distancia $k-1$ por capa; para relacionar dos posiciones a distancia n hacen falta del orden de n/k capas, mientras que la atención las conecta en una. La red recurrente necesita $O(n)$ pasos secuenciales.

Resultado. $1{,}718\cdot10^{10}$**,** $4{,}295\cdot10^{9}$ **y** $1{,}288\cdot10^{10}$**; la atención es más cara que la recurrente para** $n > 1024$**.** La ventaja de la atención es su camino de longitud uno entre cualquier par de posiciones y su paralelismo; su inconveniente, el coste cuadrático que motiva la ventana deslizante y los SSM.

Comprobación. el cociente atención/recurrente es $1{,}718 \cdot 10^{10}/4{,}295 \cdot 10^{9} = 4 = n/d$.

Solución del ejercicio 3.6

Datos. ventana dilatada: la posición i lee $i - a\delta$ con $a \in \{0, \dots, w-1\}$; L capas.

Planteamiento. inducción sobre L, describiendo el conjunto de desplazamientos alcanzables $D_L = \{i - j\}$.

Desarrollo.

1. $D_1 = \{0, \delta, \dots, (w-1)\delta\}$; el máximo es $\delta(w-1)$.
2. Si D_L es el conjunto de desplazamientos tras L capas, la capa $L+1$ suma un desplazamiento de D_1: $D_{L+1} = \{u + v : u \in D_1,\ v \in D_L\}$.
3. Por inducción, $D_L = \{\delta(a_1 + \cdots + a_L) : a_l \in \{0, \dots, w-1\}\}$, cuyo máximo es $L\delta(w-1)$, alcanzado con todos los $a_l = w-1$. La salida en i solo depende de $[i - L\delta(w-1), i]$ y el extremo sí influye.
4. Todos los elementos de D_L son múltiplos de δ. Si $\delta > 1$, las posiciones $i-1, \dots, i-\delta+1$ no influyen: el intervalo no se cubre entero.

Resultado. **el campo receptivo abarca** $L\delta(w-1)+1$ **posiciones, pero con** $\delta > 1$ **solo influyen las que distan un múltiplo de** δ**.** La estimación $L\delta w$ de Longformer es la longitud del intervalo; para cubrir los huecos, el artículo usa dilataciones distintas en distintas cabezas. ■

Comprobación. con $\delta = 1$ se recupera el teorema 3.3: $L(w-1)+1$ posiciones, todas alcanzables.

Solución del ejercicio 3.7

Datos. $\theta'_i = (\beta'_0)^{-2i/d_h}$; PI usa el ángulo $(m/s_e)\theta_i$ con $0 \le m < L' = s_e L_0$.

Planteamiento. (a) imponer la condición sobre $\theta'_{d_h/2-1}$ y despejar β'_0; (b) acotar el ángulo.

Desarrollo.

1. (a) $\theta'_0 = (\beta'_0)^0 = 1 = \theta_0$ para cualquier base: la primera condición se cumple siempre.
2. $\theta'_{d_h/2-1} = (\beta'_0)^{-(d_h-2)/d_h}$ y se exige que valga $\beta_0^{-(d_h-2)/d_h}/s_e$.
3. Elevando ambos miembros a $-d_h/(d_h-2)$ (la función $u \mapsto u^{-d_h/(d_h-2)}$ es biyectiva en $(0,\infty)$): $\beta'_0 = \beta_0\, s_e^{d_h/(d_h-2)}$, única solución.
4. (b) Con PI, el mayor ángulo del par i se alcanza en $m = L' - 1$: $(L'-1)\theta_i/s_e = (L_0 - 1/s_e)\theta_i < L_0\theta_i$.
5. En el entrenamiento, el mayor ángulo es $(L_0-1)\theta_i$; con PI, $(L_0-1/s_e)\theta_i$, que solo excede al anterior en $(1-1/s_e)\theta_i < \theta_i$, menos de un paso de posición.

Resultado. **(a) $\beta_0' = \beta_0 s_{\mathrm{e}}^{d_h/(d_h-2)}$ es la única base con esas dos propiedades; (b) con PI los ángulos quedan dentro del intervalo $[0, L_0\theta_i)$, es decir, a menos de un paso del máximo visto en el entrenamiento.** ■

Comprobación. con $d_h = 128$ y $s_{\mathrm{e}} = 4$, (a) reproduce $\beta_0' = 40\,890$ del ejercicio 3.4.

A.4 Tema 4

Solución del ejercicio 4.1

Datos. (a) $N = 8$, $k = 2$; (b) un token con una sola asignación y experto sin capacidad.

Planteamiento. proposición 4.4 y definición 4.9.

Desarrollo.

1. (a) El enrutador puede elegir cualquier experto en cualquier token y capa, de modo que los 8 deben estar en memoria; cada token usa 2.
2. Parámetros de MLP guardados/usados: $8P_E/(2P_E) = 4$. El modelo denso con los mismos activos guarda solo $2P_E$ por capa.
3. (b) El experto lleno no evalúa ese token; la contribución del MLP es nula y la salida del bloque es $h + 0 = h$.
4. El vector del token atraviesa la capa por la conexión residual con toda la información previa; solo se pierde la transformación de ese MLP en esa capa.

Resultado. **(a) la MoE guarda 4 veces más parámetros de MLP de los que usa por token; (b) el token se descarta en esa capa, pero su información sigue en el flujo residual.**

Comprobación. con $k = N$ el cociente es $N/N = 1$: sin dispersión no hay sobrecoste de memoria.

Solución del ejercicio 4.2

Datos. tres afirmaciones sobre Gemini 1.5, el Switch Transformer y DeepSeek-V3.

Planteamiento. contrastar con la cuadro 4.2 y las observaciones del tema.

Desarrollo.

(a) **Falsa.** El informe solo dice que es un Transformer MoE disperso; no publica el número de expertos [12, §3.1]. La cifra de 8 corresponde a Mixtral y Grok-1.

(b) **Verdadera.** El Switch Transformer usa enrutado top-1 [10, §2.1].

(c) **Falsa.** DeepSeek-V3 sustituye la pérdida por expertos por sesgos ajustados en línea, pero conserva una pérdida de balanceo por secuencia con un coeficiente muy pequeño [7, §2.1.2].

Resultado. **(a) falsa (no es público); (b) verdadera; (c) falsa.**

Comprobación. la clase 4 de CS336 describe el método de DeepSeek-V3 como balanceo «sin pérdida auxiliar» y advierte de que no lo es del todo [16].

Solución del ejercicio 4.3

Datos. $z = (0{,}5;\ 2;\ 1{,}2;\ 0)$, $k = 2$.

Planteamiento. elegir los dos logits mayores y aplicar la softmax entre ellos (definición 4.1); para la comparación, la softmax sobre los cuatro.

Desarrollo.

1. Orden: 2 (experto 2) $>$ 1,2 (experto 3) $>$ 0,5 $>$ 0. $\mathcal{T} = \{2, 3\}$.
2. $e^2 = 7{,}389$; $e^{1,2} = 3{,}320$; suma 10,71.
3. $g_2 = 7{,}389/10{,}71 = 0{,}6900$; $g_3 = 3{,}320/10{,}71 = 0{,}3100$. Equivalentemente, $g_2 = 1/(1 + e^{-0{,}8}) = 1/1{,}449 = 0{,}6900$.
4. Softmax sobre los cuatro: $e^{0,5} + e^2 + e^{1,2} + e^0 = 1{,}649 + 7{,}389 + 3{,}320 + 1 = 13{,}36$; $p_2 = 0{,}5532$ y $p_3 = 0{,}2486$.

Resultado. **expertos 2 y 3, con $g_2 = 0{,}6900$ y $g_3 = 0{,}3100$; sin renormalizar serían 0,5532 y 0,2486.**

Comprobación. $p_2/(p_2 + p_3) = 0{,}5532/0{,}8018 = 0{,}6900 = g_2$, como establece la proposición 4.2.

Solución del ejercicio 4.4

Datos. (a) $N = 4$, $f = (0{,}4;\ 0{,}1;\ 0{,}35;\ 0{,}15)$, $P = (0{,}35;\ 0{,}15;\ 0{,}35;\ 0{,}15)$; (b) $B_{\mathrm{t}} = 2048$, $N = 16$, $k = 1$, $c = 1{,}0$.

Planteamiento. definición 4.6 y definición 4.9.

Desarrollo.

1. Productos: $0{,}4 \cdot 0{,}35 = 0{,}14$; $0{,}1 \cdot 0{,}15 = 0{,}015$; $0{,}35 \cdot 0{,}35 = 0{,}1225$; $0{,}15 \cdot 0{,}15 = 0{,}0225$.
2. Suma: 0,3; $\mathcal{L}_{\mathrm{bal}} = 4 \cdot 0{,}3 = 1{,}2$.
3. (b) Asignaciones: $kB_{\mathrm{t}} = 2048$; reparto ideal $2048/16 = 128$; $C_{\mathrm{exp}} = 1{,}0 \cdot 128 = 128$.
4. Las 16 capacidades suman 2048, exactamente las asignaciones. Si un experto recibe más de 128 tokens, otro recibe menos; los sobrantes del primero se descartan mientras quedan plazas libres en el segundo.

Resultado. **(a) $\mathcal{L}_{\mathrm{bal}} = 1{,}2$; (b) $C_{\mathrm{exp}} = 128$, sin holgura: cualquier desequilibrio provoca descartes.**

Comprobación. en (a), f y P suman 1 y el valor está en $[1, 4]$, aunque $f \neq P$; con $f = P$ el teorema 4.7 lo garantizaría.

Solución del ejercicio 4.5

Datos. $d = 6144$, $L = 64$, $H = 48$, $n_{\mathrm{kv}} = 8$, $d_h = 128$, $N = 8$, $k = 2$, $V = 131\,072$, *embeddings* compartidos, $d_{\mathrm{ff}} = \lfloor 8d \cdot 2/3 \rfloor$, tres matrices por experto.

Planteamiento. proposición 4.4: P_0 = atención + *embeddings* + enrutadores.

Desarrollo.

1. $d_{\mathrm{ff}} = \lfloor 49\,152 \cdot 2/3 \rfloor = 32\,768$ (ya es múltiplo de 8).
2. $P_E = 3 \cdot 6144 \cdot 32\,768 = 603\,979\,776$; todos los expertos: $8 \cdot 64 \cdot P_E = 309\,237\,645\,312$.

3. Atención por capa: $2 \cdot 6144^2 + 2 \cdot 6144 \cdot 1024 = 75\,497\,472 + 12\,582\,912 = 88\,080\,384$; en 64 capas, $5\,637\,144\,576$.
4. *Embeddings* compartidos: $131\,072 \cdot 6144 = 805\,306\,368$; enrutadores: $64 \cdot 6144 \cdot 8 = 3\,145\,728$.
5. $N_{\text{tot}} = 309\,237\,645\,312 + 5\,637\,144\,576 + 805\,306\,368 + 3\,145\,728 = 315\,683\,241\,984$.
6. $N_{\text{act}} = 2{\cdot}64{\cdot}P_E + 5\,637\,144\,576 + 805\,306\,368 + 3\,145\,728 = 77\,309\,411\,328 + 6\,445\,596\,672 = 83\,755\,008\,000$.
7. Fracción activa: $83\,755\,008\,000/315\,683\,241\,984 = 26{,}53\,\%$.

Resultado. $P_E = 603\,979\,776$**;** $N_{\text{tot}} \approx 3{,}157 \cdot 10^{11}$ **y** $N_{\text{act}} \approx 8{,}376 \cdot 10^{10}$ **(26,53 %).** El total supera en un 0,54 % la cifra redondeada del archivo `README` ($3{,}14 \cdot 10^{11}$); xAI no explica el redondeo, de modo que la diferencia no puede atribuirse con seguridad a ninguna parte del modelo. La fracción activa es coherente con «una cuarta parte de los pesos por token».

Comprobación. los expertos son el 97,96 % de los parámetros,

$$309\,237\,645\,312/315\,683\,241\,984 = 0{,}9796,$$

y por eso $N_{\text{act}}/N_{\text{tot}}$ queda cerca de $k/N = 0{,}25$.

Solución del ejercicio 4.6

Datos. f, P distribuciones sobre $\{1, \dots, N\}$.

Planteamiento. acotar $\sum_i f_i P_i$ por debajo con la no negatividad y por arriba con $P_i \le \max_j P_j \le 1$.

Desarrollo.

1. $f_i P_i \ge 0$, luego $\mathcal{L}_{\text{bal}} \ge 0$; se alcanza con $f = (1, 0, \dots, 0)$ y $P = (0, 1, 0, \dots, 0)$.
2. $\sum_i f_i P_i \le \max_j P_j \sum_i f_i \le 1$, luego $\mathcal{L}_{\text{bal}} \le N$; se alcanza con $f = P = (1, 0, \dots, 0)$.
3. En la práctica f y P no son independientes: con top-1, un token se asigna al experto i solo si $p_i(x)$ es la mayor de las N probabilidades, y entonces $p_i(x) \ge 1/N$. Por tanto $P_i \ge \frac{1}{T} \sum_{x \to i} p_i(x) \ge f_i/N$.
4. De ahí, $\mathcal{L}_{\text{bal}} = N \sum_i f_i P_i \ge \sum_i f_i^2 \ge 1/N > 0$ (Cauchy–Schwarz, como en el teorema 4.7): el valor 0 no es alcanzable por un enrutador real.

Resultado. $0 \le \mathcal{L}_{\text{bal}} \le N$ **para distribuciones arbitrarias, con ambas cotas alcanzables; pero, si** f **procede del enrutador,** $\mathcal{L}_{\text{bal}} \ge 1/N$**, y el valor de referencia es el del reparto uniforme,** $\mathcal{L}_{\text{bal}} = 1$**.** ■

Comprobación. con $N = 4$ y reparto uniforme, $\sum_i f_i^2 = 1/4$ y $\mathcal{L}_{\text{bal}} = 1 \ge 1/4$.

Solución del ejercicio 4.7

Datos. T tokens, top-1, $\mathcal{L}_{\text{bal}} = N \sum_i f_i P_i$ con $P_i = \frac{1}{T} \sum_x p_i(x)$; (b) $T = 8$, $N = 4$, $\alpha = 10^{-2}$, asignaciones $(4, 2, 1, 1)$.

Planteamiento. f es constante; P_i depende linealmente de $p_i(x)$.

Desarrollo.

1. (a) $\partial P_j / \partial p_i(x) = \frac{1}{T}$ si $j = i$ y 0 en otro caso.

2. $\partial(\alpha\mathcal{L}_{\text{bal}})/\partial p_i(x) = \alpha N f_i \cdot \frac{1}{T} = \frac{\alpha N}{T} \cdot \frac{1}{T} \sum_{x'} \mathbf{1}[\arg\max_j p_j(x') = i] = \frac{\alpha N}{T^2} \sum_{x'} \mathbf{1}[\cdot]$.
3. (b) $\alpha N/T^2 = 0{,}04/64 = 6{,}25 \cdot 10^{-4}$; derivadas: $4 \cdot 6{,}25 \cdot 10^{-4} = 2{,}5 \cdot 10^{-3}$, $1{,}25 \cdot 10^{-3}$, $6{,}25 \cdot 10^{-4}$ y $6{,}25 \cdot 10^{-4}$.
4. (c) El descenso de gradiente reduce $p_i(x)$ en proporción a esa derivada: el experto con más asignaciones ve bajar su probabilidad con más fuerza. Como la softmax suma 1, la masa pasa a los demás, y en el lote siguiente el reparto tiende a igualarse.

Resultado. **la derivada es $\alpha N/T^2$ por el número de tokens asignados al experto; en el ejemplo,** $(2{,}5;\ 1{,}25;\ 0{,}625;\ 0{,}625) \cdot 10^{-3}$**.** Es una realimentación negativa: la penalización crece con el tráfico. ■

Comprobación. la derivada del experto 1 es cuatro veces la de los expertos 3 y 4, igual que su número de asignaciones (4/1).

A.5 Tema 5

Solución del ejercicio 5.1

Datos. S4 con $(\bar{A}, \bar{B}, C)$ fijos; Mamba con B_t, C_t y Δ_t dependientes de la entrada.

Planteamiento. la forma convolucional exige un núcleo independiente de t (proposición 5.5); el barrido exige asociatividad (proposición 5.10).

Desarrollo.

1. En S4, $y_t = \sum_j C\bar{A}^j\bar{B}x_{t-j}$ con un núcleo $\bar{K}_j = C\bar{A}^j\bar{B}$ fijo: es una convolución y se calcula en paralelo.
2. En Mamba, el coeficiente de x_{t-j} es $C_t\bar{A}_t \cdots \bar{A}_{t-j+1}\bar{B}_{t-j}$, que depende de t: no existe un núcleo común.
3. La recurrencia sigue siendo afín en el estado: cada paso es $h \mapsto \bar{A}_t h + \bar{B}_t x_t$. La composición de afines es afín y asociativa (proposición 5.10), y un barrido paralelo calcula todos los prefijos con profundidad $O(\log T)$.

Resultado. **S4 es invariante en el tiempo y admite convolución; Mamba no, pero su linealidad en el estado permite el barrido paralelo.**

Comprobación. si en Mamba se fijan $\bar{A}_t = \bar{A}$ y $\bar{B}_t = \bar{B}$, el coeficiente pasa a ser $C\bar{A}^j\bar{B}$ y se recupera el núcleo de S4.

Solución del ejercicio 5.2

Datos. $\bar{A} = 0{,}9$, $\bar{B} = C = 1$, $x = (1, 0, 0, 0, 1)$, $h_0 = 0$.

Planteamiento. recurrencia $h_t = 0{,}9h_{t-1} + x_t$; núcleo $\bar{K}_j = 0{,}9^j$.

Desarrollo.

1. $h_1 = 1$; $h_2 = 0{,}9$; $h_3 = 0{,}81$; $h_4 = 0{,}729$; $h_5 = 0{,}9 \cdot 0{,}729 + 1 = 0{,}6561 + 1 = 1{,}6561$.
2. Núcleo: $\bar{K} = (1;\ 0{,}9;\ 0{,}81;\ 0{,}729;\ 0{,}6561)$.
3. Convolución en $t = 5$: $\bar{K}_0 x_5 + \bar{K}_4 x_1 = 1 + 0{,}6561 = 1{,}6561$; para $t \le 4$ solo actúa x_1 y $y_t = \bar{K}_{t-1}$.

Resultado. $y = (1;\ 0{,}9;\ 0{,}81;\ 0{,}729;\ 1{,}6561)$ **por ambos métodos.**

Comprobación. la entrada inicial pesa $0{,}9^4 = 0{,}6561$ cuatro pasos después: la memoria decae más despacio que con $\bar{A} = 0{,}5$ en el ejemplo 5.6.

Solución del ejercicio 5.3

Datos. $a = -2$, $\beta = 1$, $\Delta = 0{,}5$.

Planteamiento. proposición 5.2; bilineal $\bar{A} = (1 + \Delta a/2)/(1 - \Delta a/2)$, $\bar{B} = \Delta\beta/(1 - \Delta a/2)$; Euler $\bar{A} = 1 + \Delta a$, $\bar{B} = \Delta\beta$.

Desarrollo.

1. Orden cero: $\bar{A} = e^{-1} = 0{,}3679$; $\bar{B} = (e^{-1} - 1)/(-2) = 0{,}6321/2 = 0{,}3161$.
2. Bilineal: $\bar{A} = (1 - 0{,}5)/(1 + 0{,}5) = 0{,}3333$; $\bar{B} = 0{,}5/1{,}5 = 0{,}3333$.
3. Euler: $\bar{A} = 1 - 1 = 0$; $\bar{B} = 0{,}5$.
4. Estabilidad para cualquier Δ: con orden cero, $\bar{A} = e^{a\Delta} \in (0, 1)$. Con la bilineal, $|1 + \Delta a/2| < |1 - \Delta a/2|$ porque $a < 0$, luego $|\bar{A}| < 1$. Con Euler, $|1 + \Delta a| < 1$ exige $\Delta < 2/|a| = 1$: con $\Delta = 1{,}5$, $\bar{A} = -2$ y la recurrencia diverge.

Resultado. **orden cero** $(0{,}3679;\ 0{,}3161)$**, bilineal** $(0{,}3333;\ 0{,}3333)$**, Euler** $(0;\ 0{,}5)$**; los dos primeros son estables para todo** $\Delta > 0$ **y Euler solo para** $\Delta < 1$**.**

Comprobación. con entrada constante $x = 1$, el equilibrio del sistema continuo es $h^* = -\beta/a = 0{,}5$, y los tres métodos lo conservan: $0{,}3161/(1 - 0{,}3679) = 0{,}5$, $0{,}3333/0{,}6667 = 0{,}5$ y $0{,}5/1 = 0{,}5$.

Solución del ejercicio 5.4

Datos. (a) $y[n] = 0{,}75y[n-1] + x[n-1]$, $x = \delta$, reposo; (b) etapas $g[n] = \frac{1}{2}g[n-1] + x[n]$ y $h[n] = \frac{1}{4}h[n-1] + g[n]$.

Planteamiento. aplicar las recurrencias desde el reposo.

Desarrollo.

1. (a) $y[0] = 0{,}75{\cdot}0 + x[-1] = 0$; $y[1] = 0 + x[0] = 1$; $y[2] = 0{,}75$; $y[3] = 0{,}5625$; $y[4] = 0{,}4219$.
2. Fórmula: $y[n] = 0{,}75^{\,n-1}$ para $n \geq 1$ e $y[0] = 0$: un SSM con $\bar{A} = 0{,}75$ y la entrada retrasada un paso.
3. (b) Primera etapa: $g = (1;\ 0{,}5;\ 0{,}25;\ 0{,}125)$.
4. Segunda etapa: $h[0] = 1$; $h[1] = 0{,}25 + 0{,}5 = 0{,}75$; $h[2] = 0{,}1875 + 0{,}25 = 0{,}4375$; $h[3] = 0{,}109\,375 + 0{,}125 = 0{,}2344$.
5. Fórmula: $2{\cdot}1 - 1 = 1$; $2{\cdot}0{,}5 - 0{,}25 = 0{,}75$; $2{\cdot}0{,}25 - 0{,}0625 = 0{,}4375$; $2{\cdot}0{,}125 - 0{,}015\,625 = 0{,}2344$.

Resultado. **(a)** $(0;\ 1;\ 0{,}75;\ 0{,}5625;\ 0{,}4219)$**,** $y[n] = 0{,}75^{\,n-1}$**; (b)** $(1;\ 0{,}75;\ 0{,}4375;\ 0{,}2344)$**, que coincide con la fórmula.**

Comprobación. la cascada es la convolución de los núcleos: $h[2] = \sum_{j=0}^{2} (1/2)^j (1/4)^{2-j} = 1/16 + 1/8 + 1/4 = 7/16 = 0{,}4375$.

Solución del ejercicio 5.5

Datos. $h_0 = 1$, $z = (1, -1, 3)$, $x = (0, 2, -1)$.

Planteamiento. teorema 5.8: $\Delta_t = \text{softplus}(z_t)$, $g_t = \sigma(z_t)$, $h_t = (1-g_t)h_{t-1} + g_t x_t$.

Desarrollo.

1. $t=1$: $\Delta_1 = \ln(1+e) = 1{,}313$; $g_1 = \sigma(1) = 0{,}7311$; $h_1 = 0{,}2689 \cdot 1 + 0{,}7311 \cdot 0 = 0{,}2689$.
2. Comprobación por orden cero en $t=1$: $\bar{A}_1 = e^{-1{,}313} = 0{,}2689$ y $\bar{B}_1 = 1 - 0{,}2689 = 0{,}7311$; $h_1 = 0{,}2689$.
3. $t=2$: $\Delta_2 = \ln(1+e^{-1}) = 0{,}3133$; $g_2 = 0{,}2689$; $h_2 = 0{,}7311 \cdot 0{,}2689 + 0{,}2689 \cdot 2 = 0{,}1966 + 0{,}5379 = 0{,}7345$.
4. $t=3$: $\Delta_3 = \ln(1+e^{3}) = 3{,}049$; $g_3 = 0{,}9526$; $h_3 = 0{,}047\,43 \cdot 0{,}7345 + 0{,}9526 \cdot (-1) = 0{,}034\,83 - 0{,}9526 = -0{,}9177$.

Resultado. los pasos son $\Delta = (1{,}313;\ 0{,}3133;\ 3{,}049)$ y las puertas, $g = (0{,}7311;\ 0{,}2689;\ 0{,}9526)$; los estados son $h = (0{,}2689;\ 0{,}7345;\ -0{,}9177)$.

Comprobación. un paso Δ_t grande hace $\bar{A}_t$ pequeño y el estado se reemplaza casi por la entrada ($t=3$: $h_3 \approx x_3 = -1$); un paso pequeño conserva el estado.

Solución del ejercicio 5.6

Datos. $y[n] = \frac{1}{2}y[n-2] + x[n]$; $\bar{A} = \begin{pmatrix} 0 & 1/2 \\ 1 & 0 \end{pmatrix}$, $\bar{B} = (1,0)^\top$, $C = (1,0)$.

Planteamiento. (a) recurrencia con $x = \delta$; (b) potencias de $\bar{A}$ sobre $\bar{B}$ y polinomio característico; (c) forma del núcleo de un SSM escalar.

Desarrollo.

1. (a) $y[0] = 1$; $y[1] = \frac{1}{2}y[-1] = 0$; $y[2] = \frac{1}{2}$; $y[3] = 0$; $y[4] = \frac{1}{4}$. Coincide con K.
2. (b) $\bar{B} = (1,0)^\top$, $C\bar{B} = 1$; $\bar{A}\bar{B} = (0,1)^\top$, $C\cdot = 0$; $\bar{A}^2\bar{B} = (\frac{1}{2},0)^\top$, $C\cdot = \frac{1}{2}$; $\bar{A}^3\bar{B} = (0,\frac{1}{2})^\top$, $C\cdot = 0$; $\bar{A}^4\bar{B} = (\frac{1}{4},0)^\top$, $C\cdot = \frac{1}{4}$.
3. $\det(\bar{A} - \lambda I) = \lambda^2 - \frac{1}{2} = 0$, luego $\lambda = \pm 1/\sqrt{2} = \pm 0{,}7071$.
4. (c) Un SSM escalar tiene $K_n = c\,\bar{a}^{\,n}\,\bar{b}$. $K_0 = cb = 1 \neq 0$ obliga a $\bar{a} = 0$ para que $K_1 = cb\,\bar{a} = 0$, pero entonces $K_2 = 0 \neq \frac{1}{2}$. Contradicción.

Resultado. $C\bar{A}^n\bar{B} = (1;\ 0;\ 0{,}5;\ 0;\ 0{,}25)$**; polos** $\pm 0{,}7071$**; un SSM escalar no puede producir ceros alternos.** ■

Comprobación. $\bar{A}^2 = \frac{1}{2}I$, de modo que cada dos pasos el estado se multiplica por $\frac{1}{2}$; los dos polos de signos opuestos se cancelan en los instantes impares.

Solución del ejercicio 5.7

Datos. $e_t = (a_t, b_t)$: $(0{,}5;\ 1)$, $(0{,}5;\ 0)$, $(0{,}5;\ 0)$, $(0{,}5;\ 2)$; $(a,b) \bullet (a',b') = (a'a,\ a'b + b')$.

Planteamiento. agrupar en árbol, como haría un barrido paralelo, y usar la proposición 5.10.

Desarrollo.

1. $e_1 \bullet e_2 = (0{,}25;\ 0{,}5 \cdot 1 + 0) = (0{,}25;\ 0{,}5)$.
2. $e_3 \bullet e_4 = (0{,}25;\ 0{,}5 \cdot 0 + 2) = (0{,}25;\ 2)$.
3. $(0{,}25;\ 0{,}5) \bullet (0{,}25;\ 2) = (0{,}0625;\ 0{,}25 \cdot 0{,}5 + 2) = (0{,}0625;\ 2{,}125)$. Luego $h_4 = 2{,}125$.
4. (b) $(1,0) \bullet (a,b) = (a \cdot 1,\ a \cdot 0 + b) = (a,b)$ y $(a,b) \bullet (1,0) = (a,\ b + 0) = (a,b)$.

5. No conmutatividad: $(0{,}5;\ 1)\bullet(1;\ 2) = (0{,}5;\ 3)$, mientras que $(1;\ 2)\bullet(0{,}5;\ 1) = (0{,}5;\ 0{,}5\cdot 2+1) = (0{,}5;\ 2)$.

Resultado. $h_4 = 2{,}125$**, como en el ejemplo 5.6;** $(1, 0)$ **es el neutro y** $\bullet$ **no es conmutativa, por lo que el barrido debe respetar el orden de los pasos.** ■

Comprobación. la primera componente, $0{,}0625 = 0{,}5^4$, es el factor por el que se multiplicaría un estado inicial no nulo h_0.

A.6 Tema 6

Solución del ejercicio 6.1

Datos. imagen 224×224; parches de 16 o de 32.

Planteamiento. proposición 6.2 y cuadro 6.1.

Desarrollo.

1. Parches: $(224/16)^2 = 196$ frente a $(224/32)^2 = 49$; cociente 4.
2. Coste de la atención: $\propto N_p^2$, se divide por $4^2 = 16$; capas lineales del Transformer: por 4.
3. Dimensión de cada parche: 768 frente a $32^2 \cdot 3 = 3072$; la proyección E crece 4 veces, pero se aplica a 4 veces menos parches.
4. Se pierde resolución espacial: cada token resume una zona 4 veces mayor y los detalles pequeños quedan dentro de un único vector.

Resultado. **4 veces menos tokens y unas 16 veces menos coste de atención, a cambio de una representación más gruesa de la imagen.**

Comprobación. el coste de la proyección inicial no cambia: $196\cdot 768\cdot d = 49\cdot 3072\cdot d = 150\,528\,d$ multiplicaciones.

Solución del ejercicio 6.2

Datos. parche 16; lados 224 y 384.

Planteamiento. $N_p = (\text{lado}/16)^2$; lineales $\propto N_p$ y atención $\propto N_p^2$.

Desarrollo.

1. $224/16 = 14$, $N_p = 196$; $384/16 = 24$, $N_p = 576$.
2. Cociente de tokens: $576/196 = 2{,}939$.
3. Atención: $2{,}939^2 = 8{,}636$.

Resultado. **196 y 576 parches; las capas lineales crecen unas** $2{,}939$ **veces y la atención unas** $8{,}636$**.**

Comprobación. $(384/224)^2 = 1{,}714^2 = 2{,}939$ y $(384/224)^4 = 8{,}636$, como predice la proposición 6.2 con $\kappa = 1{,}714$.

Solución del ejercicio 6.3

Datos. $P = 16$, lados 96 y 192, token de clase, $d = 512$, $d_{\text{ff}} = 2048$.

Planteamiento. $n = (\text{lado}/P)^2 + 1$; lineales $n(8d^2 + 4dd_{\text{ff}})$; atención $4n^2d$.

Desarrollo.

1. $96/16 = 6$, $n = 37$; $192/16 = 12$, $n = 145$.
2. Factor lineal: $8 \cdot 262\,144 + 4 \cdot 512 \cdot 2048 = 2\,097\,152 + 4\,194\,304 = 6\,291\,456$ por token.
3. $n = 37$: lineales 232 783 872; atención $4 \cdot 1369 \cdot 512 = 2\,803\,712$; total 235 587 584.
4. $n = 145$: lineales 912 261 120; atención $4 \cdot 21\,025 \cdot 512 = 43\,059\,200$; total 955 320 320.
5. Crecimiento: $955\,320\,320/235\,587\,584 = 4{,}055$. Fracción de atención: 1,190 % y 4,507 %.
6. Igualdad: $4n^2d = n \cdot 6\,291\,456 \iff n = 6\,291\,456/2048 = 3072$.

Resultado. **37 y 145 tokens; el coste por bloque crece** 4,055 **veces y la atención pasa del** 1,190 % **al** 4,507 %**; la atención dominaría a partir de** $n = 3072$ **tokens.**

Comprobación. los tokens crecen $145/37 = 3{,}919$ y la atención $(145/37)^2 = 15{,}36$; el total queda entre ambos, cerca del primero, porque dominan las capas lineales.

Solución del ejercicio 6.4

Datos. 16 kHz; fragmentos de 30 s; desplazamiento de 10 ms; convolución con paso 2; grabación de 10 minutos; códec de 50 Hz con 8 diccionarios.

Planteamiento. contar muestras, fotogramas y vectores; para el códec, proposición 6.5.

Desarrollo.

1. (a) $16\,000 \cdot 30 = 480\,000$ muestras.
2. (b) $30/0{,}01 = 3000$ vectores de 80 canales; tras el paso 2, 1500.
3. (c) $600/30 = 20$ fragmentos; $20 \cdot 1500 = 30\,000$ vectores.
4. (d) Códec: $50 \cdot 8 = 400$ tokens por segundo; en 600 s, 240 000 tokens, 8 veces más.

Resultado. **(a)** 480 000 **muestras; (b) 3000 y 1500 vectores; (c)** 30 000 **vectores; (d)** 240 000 **tokens con el códec.** El codificador de Whisper comprime 16 veces más que la sucesión de índices del códec, pero sus vectores son continuos y no sirven como objetivo de predicción del siguiente token.

Comprobación. cada vector del codificador cubre $600\ \text{s}/30\,000 = 0{,}02$ s = 20 ms de audio, el doble del desplazamiento de 10 ms del espectrograma, como corresponde al paso 2 de la convolución.

Solución del ejercicio 6.5

Datos. ViT-Base: $d = 768$, $P = 16$, $C_{\text{in}} = 3$, $N_p = 196$, $8{,}6 \cdot 10^7$ parámetros.

Planteamiento. proposición 6.2(i) y definición 6.1.

Desarrollo.

1. E: $16^2 \cdot 3 \cdot 768 = 768 \cdot 768 = 589\,824$.
2. E_{pos}: $(196 + 1) \cdot 768 = 151\,296$.
3. Token de clase: 768.
4. Suma: 741 888; fracción $741\,888/8{,}6 \cdot 10^7 = 0{,}8627$ %.

Resultado. $589\,824 + 151\,296 + 768 = 741\,888$ **parámetros, un** $0{,}8627\,\%$ **del modelo.** Casi todos los parámetros están en los bloques del Transformer, como en un modelo de lenguaje sin vocabulario grande.

Comprobación. cada bloque de ViT-Base tiene unos $4d^2 + 2d \cdot 3072 = 2\,359\,296 + 4\,718\,592 = 7\,077\,888$ parámetros de matriz; 12 bloques suman $8{,}49 \cdot 10^7$, que con la entrada da $\approx 8{,}6 \cdot 10^7$.

Solución del ejercicio 6.6

Datos. imagen X de lado ℓ; filtros de $P \times P \times C_{\text{in}}$; paso S.

Planteamiento. escribir la salida de la convolución en la posición (u, v) y compararla con la proyección del parche.

Desarrollo.

1. (a) Una convolución con filtro $F^{(c)} \in \mathbb{R}^{P\times P\times C_{\text{in}}}$ y paso S da en (u, v) el valor $\sum_{a,b,\kappa} F^{(c)}_{a,b,\kappa} X_{uS+a,\, vS+b,\, \kappa}$.
2. Con $S = P$, los índices $(uP + a, vP + b)$, $0 \le a, b < P$, recorren exactamente el parche (u, v), y los parches no se solapan.
3. Si $x_p^{(u,v)}$ es el parche aplanado en el mismo orden que $F^{(c)}$, el valor es $x_p^{(u,v)} \cdot \operatorname{vec}(F^{(c)})$. Tomando como columna c de E el vector $\operatorname{vec}(F^{(c)})$, la salida en (u, v) es la fila $x_p^{(u,v)} E$.
4. (b) Las esquinas superiores de las ventanas son $0, S, 2S, \ldots, (\ell - P)$ en cada eje: $(\ell - P)/S + 1$ posiciones, y en dos dimensiones, su cuadrado.
5. Con $\ell = 224$, $P = 16$ y $S = 8$: $(208/8 + 1)^2 = 27^2 = 729$.

Resultado. **(a) la incrustación por parches es una convolución con** d **filtros** $P \times P$ **y paso** P**; (b) con paso** S **hay** $((\ell - P)/S + 1)^2$ **vectores: 729 en el ejemplo.** ■

Comprobación. con $S = P = 16$, $(208/16 + 1)^2 = 14^2 = 196$, el número de parches de la cuadro 6.1.

Bibliografía

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón y S. Sanghai. GQA: Training generalized multi-query Transformer models from multi-head checkpoints. En *Proceedings of EMNLP*, 2023. https://arxiv.org/abs/2305.13245

[2] I. Beltagy, M. E. Peters y A. Cohan. Longformer: The long-document Transformer. Prepublicación, 2020. https://arxiv.org/abs/2004.05150

[3] S. Chen, S. Wong, L. Chen e Y. Tian. Extending context window of large language models via position interpolation. Prepublicación, 2023. https://arxiv.org/abs/2306.15595

[4] Carnegie Mellon University. 11-711: Advanced Natural Language Processing. Otoño de 2024. Temario: mecanismos de atención, arquitecturas Transformer, preentrenamiento y multimodalidad. https://www.phontron.com/class/anlp-fall2024/

[5] Carnegie Mellon University. 11-868: Large Language Model Systems. Primavera de 2025. Temario: entrenamiento distribuido, dispersión (MoE), compresión y sistemas de inferencia. https://llmsystem.github.io/llmsystem2025spring/

[6] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. Informe técnico, 2024. https://arxiv.org/abs/2405.04434

[7] DeepSeek-AI. DeepSeek-V3 technical report. Informe técnico, 2024. https://arxiv.org/abs/2412.19437

[8] A. Défossez, J. Copet, G. Synnaeve e Y. Adi. High fidelity neural audio compression. Prepublicación, 2022. https://arxiv.org/abs/2210.13438

[9] A. Dosovitskiy *et al.* An image is worth 16×16 words: Transformers for image recognition at scale. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.11929

[10] W. Fedus, B. Zoph y N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–40, 2022. https://arxiv.org/abs/2101.03961

[11] S. Feng, K. Singh, M. C. Frank y C. D. Manning. CS25: Transformers United, edición V6. Stanford University, primavera. Sesiones de A. Gu («On the Tradeoffs of State Space Models and Transformers»), N. Tazi («The Ultra-Scale Talk: Scaling Training to Thousands of GPUs») y V. Lin («From Language Models to Native Multimodal Intelligence»). https://web.stanford.edu/class/cs25/

[12] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Informe técnico, 2024. https://arxiv.org/abs/2403.05530

[13] A. Grattafiori *et al.* (Llama Team, AI @ Meta). The Llama 3 herd of models. Informe técnico, 2024. https://arxiv.org/abs/2407.21783

[14] A. Gu y T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. Prepublicación, 2023 (revisada en 2024). https://arxiv.org/abs/2312.00752

[15] A. Gu, K. Goel y C. Ré. Efficiently modeling long sequences with structured state spaces. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2111.00396

[16] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2025. Clases 3 («Architectures, hyperparameters»), 4 («Mixture of experts») y 10 («Inference»), transparencias y código publicados. https://github.com/stanford-cs336/spring2025-lectures

[17] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. §3.4.1 (RMSNorm), §3.4.2 (MLP SwiGLU) y §7.3 (ablaciones de arquitectura). https://github.com/stanford-cs336/assignment1-basics

[18] A. Q. Jiang *et al.* Mixtral of experts. Prepublicación, 2024. https://arxiv.org/abs/2401.04088

[19] A. Q. Jiang *et al.* Mistral 7B. Prepublicación, 2023. https://arxiv.org/abs/2310.06825

[20] O. Lieber *et al.* Jamba: A hybrid Transformer-Mamba language model. Prepublicación, 2024. Citado a través de la clase 10 de CS336 [16]. https://arxiv.org/abs/2403.19887

[21] Massachusetts Institute of Technology. 6.003: Signals and Systems. MIT OpenCourseWare, otoño de 2011. *Homework* 2, problemas 1 («Finding outputs») y 6 («Unit-sample response»). https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/

[22] B. Peng, J. Quesnelle, H. Fan y E. Shippole. YaRN: Efficient context window extension of large language models. En *International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2309.00071

[23] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever. Robust speech recognition via large-scale weak supervision. En *International Conference on Machine Learning (ICML)*, 2023. https://arxiv.org/abs/2212.04356

[24] N. Shazeer. Fast Transformer decoding: One write-head is all you need. Prepublicación, 2019. https://arxiv.org/abs/1911.02150

[25] N. Shazeer. GLU variants improve Transformer. Prepublicación, 2020. https://arxiv.org/abs/2002.05202

[26] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton y J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. En *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1701.06538

[27] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen e Y. Liu. RoFormer: Enhanced Transformer with rotary position embedding. Prepublicación, 2021. Publicado en *Neurocomputing*, 568, 2024. https://arxiv.org/abs/2104.09864

[28] H. Touvron *et al.* Llama 2: Open foundation and fine-tuned chat models. Prepublicación, 2023. https://arxiv.org/abs/2307.09288

[29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[30] xAI. Grok-1: código y pesos publicados (archivos `README.md`, `run.py` y `model.py`). 2024. Licencia Apache 2.0. https://github.com/xai-org/grok-1

[31] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund y M. Tagliasacchi. SoundStream: An end-to-end neural audio codec. Prepublicación, 2021. https://arxiv.org/abs/2107.03312

[32] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 11, «Attention Mechanisms and Transformers» (§11.6 «Self-Attention and Positional Encoding» y §11.8 «Transformers for Vision»). https://d2l.ai/

[33] B. Zhang y R. Sennrich. Root mean square layer normalization. En *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. https://arxiv.org/abs/1910.07467